



ROYAUME DU MAROC

UNIVERSITÉ HASSAN II DE CASABLANCA

ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT TECHNIQUE DE
MOHAMMEDIA

Master Ingénierie Informatique : Big Data et Cloud Computing

Mémoire de fin d'études

SmartWealth AI

Plateforme de gestion de patrimoine à architecture microservices
augmentée par un agent d'intelligence artificielle

Microservices · Spring Cloud · Kafka · LangChain4j · Angular

Réalisé par :

NACIRI Mohammed El Arabi
MOUSSAFIA Mohammed

Encadré par :

M. Mohamed Youssfi

Année universitaire 2025 / 2026

Dédicace

*À nos parents,
pour leur amour inconditionnel et leurs sacrifices,*

*À nos familles et à nos proches,
pour leur soutien constant tout au long de notre parcours,*

*À nos enseignants,
pour le savoir qu'ils nous ont transmis,*

*À Salma NACIRI, Nada ELMAHFOUDI, Youness AHASLA, Mohamed NACHIT et
Mohammed BOUDALLAA,
pour leur présence, leur soutien et les moments partagés,*

*À tous ceux qui ont cru en nous,
nous dédions ce modeste travail.*

Remerciements

Au terme de ce projet de fin d'études, nous souhaitons exprimer notre sincère reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous tenons à remercier toutes celles et ceux qui, par leurs conseils, leur disponibilité ou leur soutien, ont participé à la bonne conduite de ce projet et à son aboutissement.

Nous remercions également l'ensemble du corps professoral et administratif de notre établissement pour la qualité de la formation dispensée durant ces années de master. Les enseignements reçus nous ont permis d'acquérir les connaissances et les compétences nécessaires à la réalisation de ce projet.

Nous adressons également nos remerciements aux membres du jury pour l'attention accordée à ce travail, ainsi que pour leurs observations et leurs conseils qui contribueront à enrichir notre réflexion.

Enfin, nous exprimons notre profonde gratitude à nos familles et à nos amis pour leur soutien moral, leur patience et leurs encouragements constants tout au long de ce parcours.

À toutes et à tous, nous adressons nos remerciements les plus sincères.

Résumé

La gestion de patrimoine connaît une mutation profonde portée par la digitalisation des services financiers et l'émergence des actifs numériques. Les investisseurs particuliers disposent aujourd'hui de portefeuilles hétérogènes, répartis entre cryptomonnaies, actions et épargne, dont le suivi demeure fragmenté et dépourvu de conseil personnalisé accessible.

Le présent projet de fin d'études propose la conception et la réalisation de **Smart-Wealth AI**, une plateforme de gestion de patrimoine bâtie sur une architecture microservices et augmentée par un agent d'intelligence artificielle. La plateforme permet à l'utilisateur de centraliser ses actifs, de suivre leur valorisation en temps réel, d'effectuer des transactions et de bénéficier d'un conseil financier intelligent et contextualisé.

Sur le plan architectural, la solution repose sur un ensemble de microservices développés avec Spring Boot, fédérés par une passerelle applicative (Spring Cloud Gateway) et un annuaire de services (Eureka). La sécurité est déléguée à Keycloak, qui centralise l'authentification et l'autorisation au moyen de jetons JWT. La communication inter-services combine deux paradigmes complémentaires : le style synchrone (REST), lorsqu'une réponse immédiate est requise, et le style asynchrone événementiel (Apache Kafka), pour le découplage des traitements. Chaque service dispose de sa propre base de données PostgreSQL, conformément au principe *database per service*.

La composante intelligente constitue l'apport distinctif du projet. L'agent, construit avec LangChain4j et Spring AI, combine trois mécanismes : l'appel d'outils (*tool calling*) pour accéder aux données réelles de l'utilisateur, une mémoire conversationnelle, et une base de connaissances documentaire exploitée par la technique de génération augmentée par récupération (RAG). Le frontend, développé en Angular, offre une interface réactive et multilingue.

Mots-clés : Microservices, Spring Boot, Spring Cloud, Apache Kafka, Keycloak, JWT, LangChain4j, Spring AI, RAG, Agent IA, Angular, Gestion de patrimoine.

Abstract

Wealth management is undergoing a profound transformation driven by the digitalization of financial services and the rise of digital assets. Individual investors now hold heterogeneous portfolios spread across cryptocurrencies, stocks and savings, whose monitoring remains fragmented and lacks accessible, personalized advice.

This graduation project presents the design and implementation of **SmartWealth AI**, a wealth management platform built on a microservices architecture and augmented by an artificial intelligence agent. The platform enables users to centralize their assets, track real-time valuation, perform transactions, and benefit from intelligent, contextualized financial advice.

Architecturally, the solution relies on a set of Spring Boot microservices, federated through an application gateway (Spring Cloud Gateway) and a service registry (Eureka). Security is delegated to Keycloak, which centralizes authentication and authorization using JWT tokens. Inter-service communication combines two complementary paradigms : synchronous (REST), when an immediate response is required, and asynchronous event-driven (Apache Kafka), for decoupling. Each service owns its PostgreSQL database, following the *database per service* principle.

The intelligent component is the project's distinctive contribution. The agent, built with LangChain4j and Spring AI, combines three mechanisms : tool calling to access the user's real data, conversational memory, and a document knowledge base leveraged through Retrieval-Augmented Generation (RAG). The frontend, developed in Angular, provides a reactive and multilingual interface.

Keywords : Microservices, Spring Boot, Spring Cloud, Apache Kafka, Keycloak, JWT, LangChain4j, Spring AI, RAG, AI Agent, Angular, Wealth Management.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Introduction générale	1
1 Cadre général du projet	3
1.1 Contexte du projet	3
1.1.1 La transformation numérique de la finance personnelle	3
1.1.2 L'essor des actifs numériques	4
1.1.3 Le potentiel de l'intelligence artificielle conversationnelle	4
1.2 Étude de l'existant	4
1.2.1 Panorama des solutions actuelles	4
1.2.2 Tableau comparatif	5
1.3 Problématique	5
1.4 Organisation du développement	6
1.4.1 Une démarche modulaire	6
1.4.2 Découpage en modules fonctionnels	6
2 Étude fonctionnelle	8
2.1 Objectifs du projet	8
2.2 Identification des acteurs	8
2.3 Besoins fonctionnels	9
2.3.1 Module F1 — Gestion des utilisateurs et authentification	9
2.3.2 Module F2 — Gestion du portefeuille	9
2.3.3 Module F3 — Gestion des transactions	10
2.3.4 Module F4 — Agent d'intelligence artificielle	10
2.4 Besoins non fonctionnels	10

2.5	Modélisation des cas d'utilisation	11
2.5.1	Diagramme de cas d'utilisation global	11
2.5.2	Descriptions textuelles des cas d'utilisation	13
3	Étude technique et conception	15
3.1	Choix de l'architecture microservices	15
3.1.1	Du monolithe aux microservices	15
3.1.2	Justification du choix pour SmartWealth AI	16
3.1.3	Le principe « Database per Service »	16
3.1.4	Les composants d'infrastructure	17
3.2	Choix technologiques et justifications	17
3.2.1	L'écosystème Spring	17
3.2.2	La sécurité : Keycloak, OAuth2 et JWT	18
3.2.3	La communication : REST et Apache Kafka	19
3.2.4	La persistance : PostgreSQL	20
3.2.5	L'intelligence artificielle : LangChain4j et Spring AI	20
3.2.6	Le frontend : Angular	21
3.2.7	Les services externes	21
3.2.8	Synthèse de la pile technologique	21
3.3	Présentation approfondie des technologies	21
3.3.1	Spring Boot	22
3.3.2	Spring Cloud Gateway	23
3.3.3	Netflix Eureka	23
3.3.4	Keycloak	24
3.3.5	Apache Kafka	25
3.3.6	PostgreSQL	26
3.3.7	LangChain4j	26
3.3.8	Spring AI	27
3.3.9	Angular	28
3.3.10	Docker	28
3.4	Approfondissement des concepts architecturaux	29
3.4.1	Le patron API Gateway	29
3.4.2	Le patron Service Discovery	30
3.4.3	Le principe de responsabilité unique appliqué aux services	30
3.4.4	La cohérence à terme (Eventual Consistency)	30
3.4.5	La sécurité en profondeur	30
3.5	Conception détaillée	31
3.5.1	Diagramme d'architecture globale	31
3.5.2	Diagramme de classes	32

3.5.3	Diagrammes de séquence	34
3.5.4	Diagrammes d'activité	36
4	Implémentation de la solution	41
4.1	Environnement de développement et de déploiement	41
4.1.1	Outils de développement	41
4.1.2	Conteneurisation avec Docker	41
4.2	Module 1 : Authentification et sécurité	43
4.2.1	Principe de fonctionnement	43
4.2.2	Le flux d'authentification côté serveur	43
4.2.3	La validation des jetons à la passerelle	43
4.2.4	La gestion des jetons côté client	44
4.3	Module 2 : Gestion du portefeuille	44
4.3.1	Principe de fonctionnement	44
4.3.2	L'intégration des prix de marché	44
4.3.3	Les calculs de valorisation et de performance	45
4.3.4	La valorisation consolidée	45
4.3.5	Interface de création d'un portefeuille	46
4.3.6	Interface d'ajout d'un actif	46
4.4	Module 3 : Transactions et communication événementielle	47
4.4.1	Principe de fonctionnement	47
4.4.2	La vérification synchrone du solde	47
4.4.3	La publication de l'événement	48
4.4.4	La consommation de l'événement	48
4.4.5	Page principale des transactions	49
4.4.6	Interface d'ajout d'une transaction	49
4.4.7	Propagation de l'événement dans Kafka	50
4.5	Module 4 : Agent d'intelligence artificielle	50
4.5.1	Principe de fonctionnement	50
4.5.2	La conversation avec mémoire	51
4.5.3	L'agent à outils	51
4.5.4	La base de connaissances (RAG)	51
4.5.5	La synthèse : un agent complet	52
4.6	Interface utilisateur	53
	Conclusion générale et perspectives	55
	Webographie	57

Table des figures

2.1	Diagramme de cas d'utilisation global	12
3.1	Vue d'ensemble des technologies utilisées dans SmartWealth AI	22
3.2	Architecture technique globale de SmartWealth AI	32
3.3	Diagramme de classes	33
3.4	Diagramme de séquence — Authentification	34
3.5	Diagramme de séquence — Transaction (Kafka)	35
3.6	Diagramme de séquence — Agent IA	36
3.7	Diagramme d'activité — Traitement d'une transaction	37
3.8	Diagramme d'activité — Raisonnement de l'agent	39
4.1	Extrait du fichier docker-compose.yml	42
4.2	Interface de connexion / inscription	44
4.3	Page portefeuille avec valorisation et performance	46
4.4	Interface d'ajout d'un portefeuille (Add wallet)	46
4.5	Interface d'ajout d'un actif (Add actif)	47
4.6	Page principale des transactions	49
4.7	Interface d'ajout d'une transaction (Add transaction)	50
4.8	Transaction et visualisation de l'événement dans Kafka UI	50
4.9	Interface conversationnelle de l'agent IA	53
4.10	Tableau de bord et navigation générale	54

Liste des tableaux

1.1	Comparaison des solutions existantes	5
2.1	Besoins non fonctionnels	11
2.2	Cas d'utilisation : S'authentifier	13
2.3	Cas d'utilisation : Effectuer une transaction	13
2.4	Cas d'utilisation : Consulter l'agent IA	14
3.1	Synthèse de la pile technologique	21

Introduction générale

Le secteur financier a connu, au cours de la dernière décennie, une transformation sans précédent. La démocratisation de l'accès aux marchés, l'apparition des cryptomonnaies et la multiplication des plateformes d'investissement en ligne ont profondément modifié la manière dont les particuliers gèrent leur patrimoine. Cette évolution, si elle a ouvert de nouvelles opportunités, a également introduit une complexité inédite : un investisseur moyen répartit désormais ses avoirs entre plusieurs classes d'actifs et plusieurs plateformes, sans disposer d'une vision consolidée ni d'un accompagnement adapté.

Dans le même temps, l'ingénierie logicielle a vu s'imposer de nouveaux paradigmes architecturaux. L'architecture monolithique traditionnelle, longtemps dominante, montre ses limites face aux exigences de scalabilité, de résilience et d'évolutivité des applications modernes. L'architecture microservices s'est progressivement affirmée comme une réponse à ces défis, en décomposant les systèmes en services autonomes, faiblement couplés et déployables indépendamment.

Parallèlement, l'intelligence artificielle générative a franchi un seuil décisif. Les grands modèles de langage (LLM) ne se contentent plus de générer du texte : ils peuvent désormais raisonner, planifier et interagir avec des systèmes externes au travers d'outils. Cette capacité, qui définit ce que l'on nomme un « agent », ouvre des perspectives considérables pour l'assistance intelligente, notamment dans des domaines à forte valeur informationnelle comme la finance.

C'est à la convergence de ces trois dynamiques — la transformation de la gestion de patrimoine, la maturité de l'architecture microservices, et l'avènement des agents intelligents — que s'inscrit le présent projet. Notre objectif est de concevoir et de réaliser **SmartWealth AI**, une plateforme qui unifie la gestion d'un patrimoine diversifié, tout en offrant un conseil financier intelligent grâce à un agent d'intelligence artificielle capable d'exploiter les données réelles de l'utilisateur.

Ce projet ne se limite pas à une juxtaposition de technologies. Il vise à démontrer la maîtrise d'un ensemble cohérent de concepts d'ingénierie : la conception d'une architecture distribuée sécurisée, l'orchestration de la communication synchrone et asynchrone, l'intégration d'un système de messagerie événementielle, et la mise en œuvre d'un agent d'IA combinant outils, mémoire et récupération de connaissances.

Organisation du rapport

Ce mémoire s’articule autour de quatre chapitres :

- Le **premier chapitre**, intitulé *Cadre général du projet*, situe le contexte, présente une étude de l’existant, formule la problématique et décrit la méthodologie de conduite adoptée.
- Le **deuxième chapitre**, *Étude fonctionnelle*, identifie les acteurs du système, recense les besoins fonctionnels et non fonctionnels, et formalise les cas d’utilisation.
- Le **troisième chapitre**, *Étude technique et conception*, expose et justifie les choix technologiques, détaille l’architecture microservices et présente l’ensemble des diagrammes de conception (classes, séquence, activité).
- Le **quatrième chapitre**, *Implémentation de la solution*, décrit la réalisation concrète de chaque module fonctionnel, illustrée par des extraits de code et des captures d’écran.

Une **conclusion générale** dresse enfin le bilan du travail accompli, en souligne les apports et les limites, et esquisse les perspectives d’évolution de la plateforme.

Chapitre 1

Cadre général du projet

Introduction

Tout projet informatique d'envergure s'inscrit dans un contexte qui en justifie l'existence et en oriente les choix. Ce premier chapitre a pour vocation de poser les fondations conceptuelles de SmartWealth AI. Nous y présentons d'abord le contexte général du projet, puis nous procédons à une étude de l'existant afin de situer notre solution par rapport aux offres actuelles du marché. Nous formulons ensuite la problématique centrale, avant de décrire la méthodologie de gestion de projet retenue pour mener à bien sa réalisation.

1.1 Contexte du projet

1.1.1 La transformation numérique de la finance personnelle

La gestion de patrimoine désigne l'ensemble des activités visant à administrer, faire fructifier et transmettre les avoirs d'un individu. Historiquement réservée à une clientèle fortunée et médiatisée par des conseillers spécialisés, elle s'est massivement démocratisée sous l'effet de la digitalisation. Aujourd'hui, tout individu disposant d'un smartphone peut ouvrir un compte de courtage, acheter des actions fractionnées, ou investir dans des cryptomonnaies en quelques minutes.

Cette accessibilité nouvelle s'accompagne toutefois d'une diversification croissante des supports d'investissement. Un épargnant contemporain peut détenir simultanément un livret d'épargne, un portefeuille d'actions, des parts de fonds, et un portefeuille de cryptomonnaies réparti sur plusieurs plateformes d'échange. Cette dispersion engendre trois difficultés majeures : l'absence de vision consolidée, la complexité du suivi de la performance, et le manque de conseil global tenant compte de l'ensemble du patrimoine.

1.1.2 L'essor des actifs numériques

Les cryptomonnaies constituent une classe d'actifs particulière, caractérisée par une forte volatilité et une disponibilité permanente des marchés. Leur valorisation évolue en continu, vingt-quatre heures sur vingt-quatre, ce qui rend leur suivi manuel particulièrement fastidieux. Un investisseur souhaitant connaître la valeur actuelle de son portefeuille doit consulter les cours en temps réel et effectuer lui-même les calculs de valorisation et de performance. Cette contrainte plaide en faveur d'outils automatisés capables d'agréger les données de marché et de les restituer de manière consolidée.

1.1.3 Le potentiel de l'intelligence artificielle conversationnelle

L'émergence des agents d'intelligence artificielle transforme la nature même de l'assistance logicielle. Là où un outil traditionnel se contente d'afficher des données, un agent intelligent peut interpréter une question formulée en langage naturel, aller chercher les informations pertinentes, raisonner sur celles-ci et formuler une recommandation argumentée. Appliquée à la gestion de patrimoine, cette capacité permet d'envisager un conseiller virtuel accessible en permanence, capable d'analyser le portefeuille réel de l'utilisateur et de le guider dans ses décisions.

1.2 Étude de l'existant

1.2.1 Panorama des solutions actuelles

Le marché des outils de gestion financière personnelle peut être segmenté en plusieurs catégories, chacune répondant partiellement aux besoins des investisseurs.

Les plateformes de courtage

Les plateformes de courtage (*brokers*) permettent l'achat et la vente d'actifs financiers. Elles offrent généralement une interface de trading performante et un accès direct aux marchés. Cependant, elles sont le plus souvent spécialisées dans une classe d'actifs particulière — actions pour les unes, cryptomonnaies pour les autres — et n'offrent qu'un conseil limité, voire inexistant. L'utilisateur qui diversifie ses placements doit donc multiplier les comptes et les interfaces.

Les agrégateurs de patrimoine

Les agrégateurs consolident les informations financières provenant de plusieurs sources afin d'offrir une vue d'ensemble. Ils répondent au besoin de centralisation, mais présentent deux limites importantes : d'une part, ils sont souvent passifs, se contentant d'afficher des

données sans permettre d’agir ; d’autre part, le conseil qu’ils proposent, lorsqu’il existe, demeure statique et générique, sans adaptation fine au profil de l’utilisateur.

Les robo-advisors

Les *robo-advisors* automatisent la gestion et le conseil en s’appuyant sur des algorithmes. Ils construisent et rééquilibrent des portefeuilles selon le profil de risque déclaré par l’utilisateur. Leur principal atout est l’automatisation ; leur principale faiblesse réside dans la rigidité de leurs règles et l’absence d’interaction conversationnelle. L’utilisateur ne peut généralement pas dialoguer avec le système ni obtenir des explications contextualisées.

1.2.2 Tableau comparatif

Le tableau suivant synthétise les forces et faiblesses des différentes catégories de solutions au regard des besoins identifiés.

Solution	Multi-actifs	Action	Temps réel	Conseil IA
Courtage	Non	Oui	Oui	Non
Agrégateur	Oui	Non	Partiel	Non
Robo-advisor	Partiel	Oui	Partiel	Règles figées
SmartWealth AI	Oui	Oui	Oui	Agent IA

TABLE 1.1 – Comparaison des solutions existantes

Constat : aucune des solutions existantes ne combine simultanément la centralisation multi-actifs, la capacité d’action en temps réel et un conseil conversationnel intelligent fondé sur les données réelles de l’utilisateur. C’est précisément ce vide que SmartWealth AI ambitionne de combler.

1.3 Problématique

À la lumière de l’étude de l’existant, la problématique du projet peut être formulée autour de trois axes interdépendants.

1. **La centralisation.** Comment concevoir une plateforme unique capable d’agréger des actifs de natures différentes (cryptomonnaies, actions, épargne) tout en préservant la cohérence et l’intégrité des données ?
2. **L’intelligence.** Comment offrir un conseil financier personnalisé, en langage naturel et en temps réel, qui s’appuie non pas sur des règles figées mais sur une analyse des données réelles de l’utilisateur ?

3. **La robustesse.** Comment garantir que la plateforme soit sécurisée, scalable et résiliente, afin de supporter la montée en charge et de préserver la confidentialité de données financières sensibles ?

Répondre simultanément à ces trois exigences impose des choix architecturaux structurants. Une architecture monolithique atteindrait rapidement ses limites en matière de scalabilité et de maintenabilité. C'est pourquoi nous avons opté pour une architecture microservices, complétée par une messagerie événementielle et une couche de sécurité centralisée, le tout couronné par un agent d'intelligence artificielle. Les chapitres suivants détaillent et justifient ces choix.

1.4 Organisation du développement

1.4.1 Une démarche modulaire

La réalisation d'un système aussi composite que SmartWealth AI, qui articule plusieurs services indépendants et des technologies variées, appelle une organisation rigoureuse du développement. Nous avons opté pour une **démarche modulaire**, consistant à construire la plateforme module par module, chaque module correspondant à un domaine fonctionnel cohérent et à un service autonome.

Cette organisation se justifie par plusieurs considérations. D'abord, elle permet de concevoir, développer et valider chaque module de manière indépendante, réduisant ainsi la complexité globale à un instant donné. Ensuite, elle épouse naturellement l'architecture microservices : chaque service constituant une unité autonome, il se prête idéalement à un développement isolé, testé puis intégré à l'ensemble. Enfin, cette approche facilite la vérification progressive : chaque brique est validée avant que la suivante ne soit entreprise, ce qui limite le risque d'accumulation d'erreurs.

1.4.2 Découpage en modules fonctionnels

Le développement de la plateforme s'est structuré autour de quatre modules fonctionnels, construits dans un ordre reflétant leurs dépendances :

- **Module Fondations et authentification** : mise en place de l'infrastructure (passerelle, annuaire, sécurité) et du service de gestion des utilisateurs. Ce module constitue le socle sur lequel reposent tous les autres.
- **Module Gestion du portefeuille** : développement du service de portefeuille, intégration des prix de marché et calcul de la valorisation.
- **Module Transactions et messagerie événementielle** : réalisation du service de transactions et de la communication asynchrone via Kafka.

- **Module Agent d'intelligence artificielle** : conception et intégration de l'agent conversationnel, de ses outils et de sa base de connaissances.

L'ordre de construction n'est pas arbitraire : le module d'authentification précède les autres car ils en dépendent pour la sécurisation des accès ; le module de portefeuille précède celui des transactions, ces dernières agissant sur les portefeuilles ; enfin, l'agent d'intelligence artificielle vient en dernier, car il exploite les données produites par l'ensemble des modules précédents.

Conclusion

Ce premier chapitre a permis de cerner le contexte dans lequel s'inscrit SmartWealth AI, de positionner la solution par rapport à l'existant et de formuler la problématique à laquelle elle répond. Il a également présenté l'organisation modulaire retenue pour le développement du projet. Le chapitre suivant approfondit l'analyse en formalisant les besoins fonctionnels et non fonctionnels de la plateforme.

Chapitre 2

Étude fonctionnelle

Introduction

L'étude fonctionnelle constitue une étape déterminante dans le cycle de vie d'un projet logiciel. Elle traduit les intentions générales en exigences précises et vérifiables. Ce chapitre identifie les acteurs qui interagissent avec le système, recense les besoins fonctionnels et non fonctionnels, puis formalise ces besoins au moyen de cas d'utilisation, conformément au formalisme UML.

2.1 Objectifs du projet

Les objectifs de SmartWealth AI se déclinent en quatre axes majeurs, qui structurent l'ensemble du travail de conception et de réalisation :

1. **Concevoir une architecture microservices** scalable, modulaire et faiblement couplée, garantissant l'indépendance de déploiement et d'évolution de chaque service.
2. **Assurer une sécurité robuste** par une authentification et une autorisation centralisées, fondées sur des standards éprouvés (OAuth2, JWT).
3. **Mettre en œuvre une communication événementielle** fiable et découplée, au moyen d'un système de messagerie distribué.
4. **Intégrer un agent d'intelligence artificielle** capable de raisonner, d'utiliser des outils pour accéder aux données réelles et d'exploiter une base de connaissances.

2.2 Identification des acteurs

Un acteur représente une entité externe (un utilisateur humain ou un système tiers) qui interagit avec le système. L'analyse du domaine fait ressortir deux acteurs principaux.

Utilisateur

L'utilisateur standard représente le client de la plateforme. Il gère son profil, crée et administre ses portefeuilles, ajoute des actifs, effectue des transactions (dépôts et retraits), consulte la valorisation de son patrimoine et dialogue avec l'agent d'intelligence artificielle pour obtenir des conseils.

Administrateur

L'administrateur hérite de l'ensemble des droits de l'utilisateur standard et dispose, en outre, de privilèges étendus. Il peut superviser les utilisateurs de la plateforme et accéder aux fonctions de surveillance (*monitoring*) du système. Cette relation d'héritage traduit le fait que l'administrateur est, avant tout, un utilisateur doté de responsabilités supplémentaires.

2.3 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit rendre à ses utilisateurs. Ils sont organisés en quatre modules cohérents, correspondant aux grands domaines fonctionnels de la plateforme.

2.3.1 Module F1 — Gestion des utilisateurs et authentification

Ce module regroupe l'ensemble des fonctionnalités liées à l'identité et à l'accès :

- Inscription d'un nouvel utilisateur ;
- Connexion au moyen d'identifiants (courriel et mot de passe) ;
- Génération et renouvellement des jetons d'accès (JWT et *refresh token*) ;
- Gestion des rôles (utilisateur, administrateur) et contrôle d'accès associé ;
- Consultation et mise à jour du profil personnel ;
- Déconnexion et invalidation de la session.

2.3.2 Module F2 — Gestion du portefeuille

Ce module couvre la gestion des avoirs de l'utilisateur :

- Création de portefeuilles de différents types (cryptomonnaies, actions, épargne) ;
- Ajout d'actifs au sein d'un portefeuille, avec saisie de la quantité et du prix moyen d'achat ;
- Récupération automatique des prix de marché en temps réel depuis une source externe ;
- Calcul de la valorisation de chaque actif, de chaque portefeuille et du patrimoine global ;

- Calcul de la plus-value ou moins-value, en valeur absolue et en pourcentage ;
- Consultation d'une synthèse consolidée du patrimoine.

2.3.3 Module F3 — Gestion des transactions

Ce module gère les mouvements financiers :

- Réalisation de dépôts (approvisionnement d'un portefeuille) ;
- Réalisation de retraits, sous réserve de solde suffisant ;
- Vérification synchrone du solde avant tout retrait ;
- Mise à jour asynchrone du solde du portefeuille à la suite d'une transaction ;
- Consultation de l'historique des transactions.

2.3.4 Module F4 — Agent d'intelligence artificielle

Ce module constitue la composante intelligente de la plateforme :

- Dialogue conversationnel avec mémoire des échanges au sein d'une session ;
- Analyse du portefeuille réel de l'utilisateur au moyen d'outils invoqués par l'agent ;
- Calcul et interprétation d'un score de risque ;
- Consultation d'une base de connaissances financière (définitions, règles, conseils) ;
- Formulation de recommandations personnalisées et argumentées.

2.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues du système, indépendamment des services rendus. Ils sont déterminants pour la satisfaction des utilisateurs et la pérennité de la solution.

Exigence	Description
Sécurité	Authentification centralisée, chiffrement des communications, protection des accès par jetons JWT, gestion fine des rôles.
Scalabilité	Possibilité de mettre à l'échelle chaque service indépendamment en fonction de la charge.
Découplage	Réduction des dépendances entre services grâce à la communication événementielle.
Performance	Valorisation en temps quasi réel ; réponses de l'agent en quelques secondes.
Disponibilité	Résilience aux pannes : la défaillance d'un service ne doit pas entraîner celle de l'ensemble.
Maintenabilité	Séparation claire des responsabilités ; un service correspond à un domaine métier.
Portabilité	Conteneurisation des services pour un déploiement homogène.
Ergonomie	Interface réactive, intuitive et multilingue.

TABLE 2.1 – Besoins non fonctionnels

2.5 Modélisation des cas d'utilisation

2.5.1 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation offre une représentation synthétique des interactions entre les acteurs et le système. Il met en évidence les fonctionnalités accessibles à chaque acteur ainsi que la relation d'héritage entre l'administrateur et l'utilisateur.

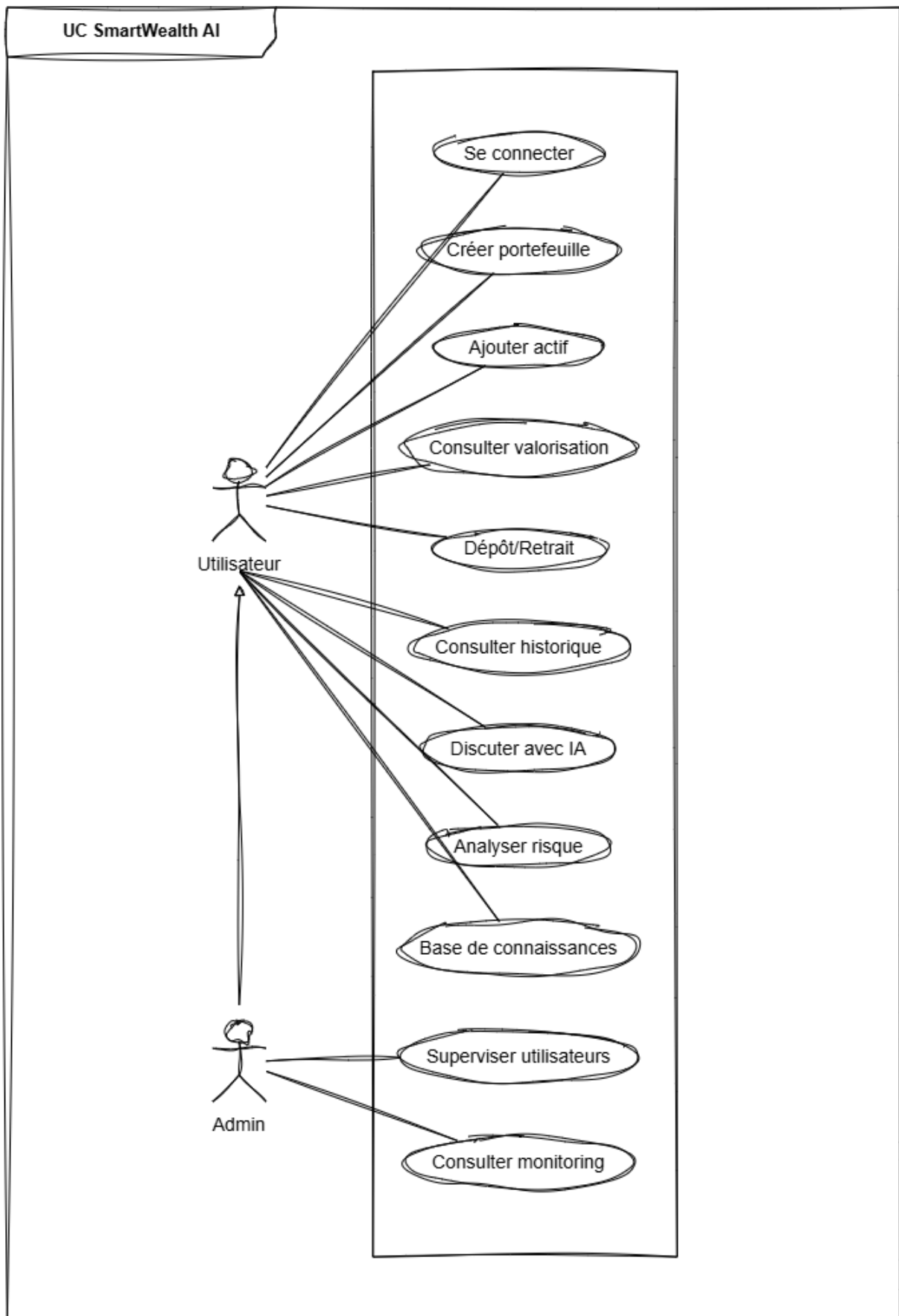


FIGURE 2.1 – Diagramme de cas d'utilisation global

2.5.2 Descriptions textuelles des cas d'utilisation

Afin de préciser le déroulement de chaque interaction, nous détaillons ci-après les principaux cas d'utilisation sous forme de fiches descriptives.

Cas d'utilisation : S'authentifier

Titre	S'authentifier
Acteur principal	Utilisateur
Pré-condition	L'utilisateur possède un compte valide.
Scénario nominal	1. L'utilisateur saisit son courriel et son mot de passe. 2. Le système transmet les identifiants au service d'authentification. 3. Le service valide les identifiants auprès de Keycloak. 4. Un jeton d'accès et un jeton de rafraîchissement sont retournés. 5. L'utilisateur accède à son tableau de bord.
Scénario alternatif	Si les identifiants sont invalides, le système affiche un message d'erreur et refuse l'accès.
Post-condition	L'utilisateur est authentifié et dispose d'une session active.

TABLE 2.2 – Cas d'utilisation : S'authentifier

Cas d'utilisation : Effectuer une transaction

Titre	Effectuer une transaction
Acteur principal	Utilisateur
Pré-condition	L'utilisateur est authentifié et possède au moins un portefeuille.
Scénario nominal	1. L'utilisateur sélectionne un portefeuille et saisit une transaction. 2. Pour un retrait, le système vérifie le solde disponible. 3. La transaction est enregistrée. 4. Un événement est publié sur le bus de messagerie. 5. Le solde du portefeuille est mis à jour de manière asynchrone.
Scénario alternatif	Si le solde est insuffisant pour un retrait, la transaction est refusée avec un message explicite.
Post-condition	La transaction est enregistrée et le solde est actualisé.

TABLE 2.3 – Cas d'utilisation : Effectuer une transaction

Cas d'utilisation : Consulter l'agent IA

Titre	Consulter l'agent d'intelligence artificielle
Acteur principal	Utilisateur
Pré-condition	L'utilisateur est authentifié.
Scénario nominal	1. L'utilisateur pose une question en langage naturel. 2. L'agent détermine s'il doit invoquer un outil. 3. Le cas échéant, il récupère les données réelles du portefeuille. 4. Il consulte la base de connaissances si nécessaire. 5. Il formule une réponse argumentée.
Scénario alternatif	Si l'information demandée ne figure pas dans la base de connaissances, l'agent l'indique honnêtement.
Post-condition	L'utilisateur reçoit une réponse contextualisée.

TABLE 2.4 – Cas d'utilisation : Consulter l'agent IA

Conclusion

Ce chapitre a formalisé les besoins de la plateforme, tant fonctionnels que non fonctionnels, et les a traduits en cas d'utilisation détaillés. Cette analyse fonctionnelle constitue le socle sur lequel repose la conception technique. Le chapitre suivant présente et justifie les choix architecturaux et technologiques, puis détaille la conception du système.

Chapitre 3

Étude technique et conception

Introduction

Après avoir défini les besoins fonctionnels, ce chapitre aborde le cœur de l'ingénierie du projet. Il présente et justifie les choix architecturaux et technologiques, en expliquant les raisons qui ont motivé chaque décision. Nous détaillons d'abord le paradigme architectural retenu, puis chacune des technologies mobilisées, avant de présenter la conception détaillée du système à travers les diagrammes UML de classes, de séquence et d'activité.

3.1 Choix de l'architecture microservices

3.1.1 Du monolithe aux microservices

Une application monolithique regroupe l'ensemble de ses fonctionnalités au sein d'une unique unité déployable. Cette approche présente des avantages indéniables pour des projets de petite taille : simplicité de développement, de test et de déploiement. Toutefois, à mesure qu'une application croît en complexité, le monolithe révèle plusieurs limites structurelles. Le couplage fort entre les composants rend toute modification risquée, car un changement local peut avoir des répercussions imprévues sur l'ensemble. La montée en charge ne peut se faire que globalement, en dupliquant l'application entière, même si une seule fonctionnalité est sollicitée. Enfin, l'ensemble du système est contraint d'utiliser une pile technologique homogène.

L'architecture microservices propose une réponse à ces limites. Elle consiste à décomposer l'application en un ensemble de services autonomes, chacun responsable d'un domaine métier précis, communiquant entre eux au moyen d'interfaces bien définies. Chaque service peut être développé, déployé, mis à l'échelle et maintenu indépendamment des autres.

3.1.2 Justification du choix pour SmartWealth AI

Le choix de l'architecture microservices pour SmartWealth AI se justifie par plusieurs facteurs directement liés à la problématique du projet :

- **Séparation des domaines.** Les fonctionnalités du projet se répartissent naturellement en domaines distincts : la gestion des utilisateurs, celle des portefeuilles, celle des transactions et l'intelligence artificielle. Cette segmentation fonctionnelle se prête idéalement à une décomposition en services.
- **Scalabilité ciblée.** Certains services, comme l'agent d'intelligence artificielle, sont plus gourmands en ressources que d'autres. L'architecture microservices permet de les mettre à l'échelle indépendamment, sans surdimensionner l'ensemble du système.
- **Résilience.** L'isolation des services garantit qu'une défaillance localisée — par exemple une indisponibilité temporaire de l'agent IA — n'affecte pas les fonctions essentielles comme la consultation du portefeuille.
- **Hétérogénéité technologique maîtrisée.** Bien que le backend soit majoritairement développé en Java, l'architecture autorise l'introduction de technologies spécifiques à certains services sans contrainte globale.

3.1.3 Le principe « Database per Service »

Un principe fondamental de l'architecture microservices est l'appropriation des données par chaque service (*database per service*). Selon ce principe, chaque microservice possède sa propre base de données et en est le seul propriétaire ; aucun autre service n'y accède directement.

Dans SmartWealth AI, ce principe est appliqué rigoureusement : le service de gestion des utilisateurs, celui des portefeuilles et celui des transactions disposent chacun de leur propre base PostgreSQL, respectivement `userdb`, `portfoliodb` et `transactiondb`. Aucune table n'est partagée. Les services sont reliés uniquement par des identifiants universels (UUID) et par les événements échangés via le bus de messagerie.

Conséquence de conception : l'absence de clés étrangères entre services impose de repenser la notion d'intégrité référentielle. Là où un monolithe garantirait la cohérence par des contraintes de base de données, l'architecture microservices s'appuie sur la cohérence à terme (*eventual consistency*), assurée par la propagation des événements. Ce compromis, exigé par le découplage, est assumé et constitue un point d'attention majeur de la conception.

3.1.4 Les composants d'infrastructure

Une architecture microservices ne se réduit pas à un ensemble de services métier ; elle requiert des composants d'infrastructure qui assurent leur coordination.

La passerelle applicative (API Gateway)

La passerelle constitue le point d'entrée unique du système. Toutes les requêtes des clients transitent par elle. Elle assume plusieurs responsabilités : le routage des requêtes vers le service approprié, la validation des jetons d'authentification, et la gestion transversale de préoccupations comme la politique de partage des ressources entre origines (CORS). En centralisant ces fonctions, la passerelle décharge les services métier et simplifie leur développement.

L'annuaire de services (Service Discovery)

Dans un environnement où les services peuvent être multipliés et déplacés dynamiquement, il n'est pas viable de coder en dur les adresses réseau. L'annuaire de services résout ce problème : chaque service s'y enregistre au démarrage, et la passerelle l'interroge pour découvrir dynamiquement l'emplacement des services. Ce mécanisme confère au système sa souplesse et sa capacité de mise à l'échelle.

3.2 Choix technologiques et justifications

3.2.1 L'écosystème Spring

Le backend de SmartWealth AI repose sur l'écosystème Spring, référence majeure du développement d'applications d'entreprise en Java.

Spring Boot

Spring Boot simplifie radicalement la création d'applications Spring autonomes. Grâce à son mécanisme d'auto-configuration et à ses « starters », il réduit considérablement le code de configuration nécessaire. Chaque microservice de la plateforme est une application Spring Boot indépendante, ce qui garantit homogénéité et rapidité de développement.

Spring Cloud Gateway

Pour la passerelle, nous avons retenu Spring Cloud Gateway, solution réactive et performante, parfaitement intégrée à l'écosystème Spring. Elle offre un système de routage déclaratif fondé sur des prédicats et des filtres, et s'interface nativement avec l'annuaire de services et la sécurité.

Eureka

L'annuaire de services est assuré par Netflix Eureka, intégré à Spring Cloud. Les services s'y enregistrent automatiquement, et la passerelle exploite ces enregistrements pour un routage à équilibrage de charge.

3.2.2 La sécurité : Keycloak, OAuth2 et JWT

Pourquoi une solution dédiée à la sécurité ?

La sécurité des données financières est une exigence non négociable. Plutôt que de développer un mécanisme d'authentification propriétaire — approche coûteuse et risquée — nous avons choisi de déléguer cette responsabilité à une solution éprouvée et standardisée : Keycloak.

Keycloak

Keycloak est un serveur d'identité et de gestion des accès à code source ouvert. Il implémente les standards OAuth2 et OpenID Connect, et prend en charge la gestion des utilisateurs, des rôles, des sessions et l'émission de jetons. En centralisant l'authentification, Keycloak garantit une gestion cohérente et sécurisée des identités à travers l'ensemble des microservices.

Le jeton JWT

Le jeton JWT (*JSON Web Token*) est un standard de représentation sécurisée d'informations entre parties. Un jeton JWT se compose de trois parties : un en-tête, une charge utile (*payload*) contenant les revendications (*claims*) telles que l'identité et les rôles de l'utilisateur, et une signature cryptographique qui en garantit l'intégrité. Dans notre architecture, après authentification, le client reçoit un jeton JWT qu'il joint à chacune de ses requêtes. La passerelle et les services valident ce jeton, extraient l'identité de l'utilisateur et appliquent le contrôle d'accès correspondant.

Propagation du jeton : un point de conception notable concerne l'agent d'intelligence artificielle. Lorsqu'il doit accéder au portefeuille de l'utilisateur, l'agent propage le jeton JWT de ce dernier dans ses propres appels. Ainsi, le service de portefeuille identifie correctement l'utilisateur concerné et retourne ses données, garantissant que l'agent n'accède qu'aux informations autorisées.

3.2.3 La communication : REST et Apache Kafka

Le choix des modes de communication entre services constitue l'un des points de conception les plus structurants du projet. SmartWealth AI combine délibérément deux paradigmes complémentaires.

La communication synchrone (REST)

La communication synchrone, fondée sur le protocole HTTP et le style REST, est employée lorsque l'appelant a besoin d'une réponse immédiate pour poursuivre son traitement. C'est le cas, par exemple, lors de la vérification du solde d'un portefeuille avant d'autoriser un retrait : le service de transactions doit connaître le solde avant de décider. C'est également le mode utilisé par l'agent IA pour lire, en temps réel, les données du portefeuille de l'utilisateur.

La communication asynchrone (Apache Kafka)

La communication asynchrone est employée lorsqu'aucune réponse immédiate n'est requise et que l'on souhaite découpler les services. Apache Kafka est une plateforme de diffusion d'événements distribuée, conçue pour le traitement de flux de données à haut débit.

Le fonctionnement de Kafka repose sur quelques concepts clés. Un **producteur** publie des messages sur un **sujet** (*topic*), qui joue le rôle de canal thématique. Un **consommateur** s'abonne à un sujet et traite les messages qui y sont publiés. Les messages sont conservés de manière durable dans un **broker**, et non supprimés après lecture, ce qui permet à plusieurs consommateurs de les lire indépendamment et autorise la relecture. La notion de **groupe de consommateurs** permet de répartir la charge : au sein d'un même groupe, chaque message n'est traité que par un seul consommateur.

Distinction fondamentale avec une file d'attente classique : contrairement à une file de messages traditionnelle où le message est supprimé après consommation, Kafka conserve les messages selon une politique de rétention. Cette caractéristique autorise plusieurs consommateurs indépendants et permet la relecture des événements, propriétés essentielles pour l'extensibilité et la résilience du système.

Application dans SmartWealth AI

Dans notre plateforme, lorsqu'une transaction est effectuée, le service de transactions publie un événement sur le sujet **transaction-events**. Le service de portefeuille, abonné à ce sujet, consomme l'événement et met à jour le solde du portefeuille concerné. Les deux services ne se connaissent pas directement : le producteur ignore l'existence des

consommateurs. Ce découplage confère au système une grande souplesse — de nouveaux consommateurs (notifications, audit, etc.) pourraient s’abonner au même flux sans modifier le producteur.

3.2.4 La persistance : PostgreSQL

Pour la persistance des données, nous avons retenu PostgreSQL, système de gestion de base de données relationnelle à code source ouvert, reconnu pour sa robustesse, sa conformité aux standards et sa richesse fonctionnelle. Chaque service métier dispose de sa propre instance, conformément au principe *database per service*. L’accès aux données est réalisé au moyen de JPA (*Java Persistence API*) et de son implémentation Hibernate, qui assurent le mapping objet-relationnel et affranchissent le développeur de l’écriture manuelle des requêtes courantes.

3.2.5 L’intelligence artificielle : LangChain4j et Spring AI

La composante d’intelligence artificielle mobilise deux bibliothèques complémentaires de l’écosystème Java.

LangChain4j

LangChain4j est une bibliothèque Java dédiée à la construction d’applications fondées sur les grands modèles de langage. Elle offre des abstractions de haut niveau pour l’appel d’outils, la gestion de la mémoire et la mise en œuvre de la génération augmentée par récupération. Nous l’avons retenue pour la construction de l’agent principal, en raison de la maturité de son mécanisme d’appel d’outils (*tool calling*), qui constitue le cœur du comportement agentique.

Spring AI

Spring AI est le projet officiel de l’écosystème Spring pour l’intégration de l’intelligence artificielle. Son intégration naturelle avec Spring Boot et sa gestion simple de la mémoire conversationnelle en font un choix pertinent pour l’assistant conversationnel de la plateforme.

Justification de la coexistence des deux bibliothèques

Le choix d’employer les deux bibliothèques répond à une logique de complémentarité : chacune est mobilisée là où elle excelle. Spring AI assure l’assistant conversationnel avec mémoire, tandis que LangChain4j prend en charge l’agent à outils et la base de connaissances. Cette coexistence, au sein d’un même service, illustre une approche pragmatique consistant à sélectionner le meilleur outil pour chaque tâche.

3.2.6 Le frontend : Angular

L'interface utilisateur est développée avec Angular, framework structurant pour la construction d'applications web monopages. Angular offre un typage statique grâce à TypeScript, une architecture à composants favorisant la réutilisabilité, et un système de réactivité moderne fondé sur les signaux (*signals*). L'internationalisation est assurée par la bibliothèque ngx-translate, permettant une interface multilingue.

3.2.7 Les services externes

La plateforme s'appuie sur deux services externes. L'API CoinGecko fournit les cours des cryptomonnaies en temps réel, indispensables au calcul de la valorisation. L'API d'OpenAI met à disposition le grand modèle de langage qui sous-tend l'agent d'intelligence artificielle.

3.2.8 Synthèse de la pile technologique

Catégorie	Technologies retenues
Frontend	Angular, TypeScript, Signals, ngx-translate
Backend	Spring Boot, Spring Cloud Gateway, Eureka, Spring Security
Persistance	PostgreSQL, JPA / Hibernate
Messagerie	Apache Kafka
Sécurité	Keycloak, OAuth2, JWT
Intelligence artificielle	LangChain4j, Spring AI, OpenAI
Services externes	CoinGecko (prix), OpenAI (LLM)
Outils	IntelliJ IDEA, Postman, Docker, Git

TABLE 3.1 – Synthèse de la pile technologique

3.3 Présentation approfondie des technologies

Cette section approfondit la présentation de chaque technologie majeure mobilisée dans le projet. Pour chacune, nous exposons sa nature, ses principaux atouts, et nous justifions son adoption en la comparant aux alternatives envisageables. Cette démarche comparative vise à démontrer que chaque choix résulte d'une réflexion argumentée plutôt que d'une simple habitude.

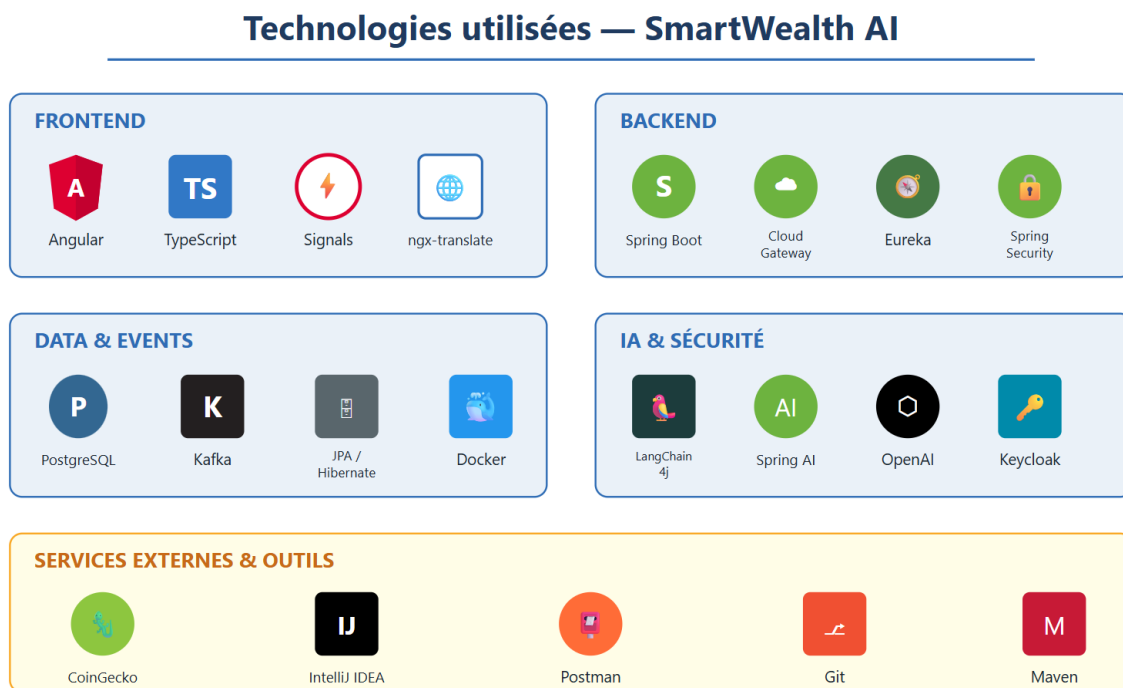


FIGURE 3.1 – Vue d’ensemble des technologies utilisées dans SmartWealth AI

3.3.1 Spring Boot

Présentation

Spring Boot est un cadriciel (*framework*) Java qui repose sur le socle Spring. Il a pour vocation de simplifier la création d’applications autonomes et prêtes pour la production. Son principe directeur est la « convention plutôt que la configuration » : il fournit des configurations par défaut sensées, que le développeur ne surcharge qu’en cas de besoin.

Avantages

- **Auto-configuration** : Spring Boot détecte les bibliothèques présentes et configure automatiquement les composants correspondants, réduisant drastiquement le code d’amorçage.
- **Serveur embarqué** : chaque application embarque son propre serveur (Tomcat, Netty), ce qui la rend exécutable de manière autonome, sans déploiement dans un conteneur externe — propriété idéale pour les microservices.
- **Écosystème riche** : l’existence de « starters » thématiques (données, sécurité, messagerie) accélère l’intégration de fonctionnalités complexes.
- **Communauté et maturité** : Spring est l’un des cadriciels les plus utilisés au monde, gage de stabilité, de documentation abondante et de pérennité.

Alternatives et justification du choix

Plusieurs cadriceis Java concurrents existent, notamment Quarkus et Micronaut, tous deux optimisés pour les temps de démarrage rapides et la faible empreinte mémoire, en particulier dans un contexte de conteneurisation. Cependant, Spring Boot a été retenu pour trois raisons déterminantes. D’abord, sa maturité et l’étendue de son écosystème garantissent la disponibilité de solutions éprouvées pour chacun de nos besoins (passerelle, découverte, sécurité, messagerie). Ensuite, l’intégration native avec Spring Cloud offre une cohérence architecturale que les alternatives ne procurent pas au même degré. Enfin, la richesse de la documentation et de la communauté constitue un atout précieux dans un contexte de projet académique où la montée en compétence rapide est essentielle.

3.3.2 Spring Cloud Gateway

Présentation

Spring Cloud Gateway est la solution de passerelle applicative de l’écosystème Spring. Bâtie sur un modèle réactif et non bloquant, elle assure le routage des requêtes entrantes vers les services appropriés, ainsi que l’application de filtres transversaux.

Avantages

- **Modèle réactif** : fondée sur Spring WebFlux, la passerelle gère un grand nombre de connexions simultanées avec une consommation de ressources maîtrisée.
- **Routage déclaratif** : les routes se définissent à l’aide de prédicats (chemin, en-tête, méthode) et de filtres, de manière lisible et maintenable.
- **Intégration native** : la passerelle s’interface directement avec l’annuaire Eureka pour le routage à équilibrage de charge, et avec la sécurité Spring pour la validation des jetons.

Alternatives et justification du choix

La principale alternative historique était Netflix Zuul. Toutefois, Zuul repose sur un modèle bloquant de première génération, aujourd’hui déprécié au profit de Spring Cloud Gateway. D’autres solutions, comme les passerelles Kong ou Traefik, opèrent au niveau de l’infrastructure et auraient introduit une dépendance externe supplémentaire. Le choix de Spring Cloud Gateway s’imposait par sa cohérence avec le reste de la pile et par son modèle réactif moderne.

3.3.3 Netflix Eureka

Présentation

Eureka est un service d'annuaire (*service registry*) qui permet aux microservices de se déclarer et de se découvrir mutuellement de manière dynamique. Chaque service s'enregistre auprès d'Eureka au démarrage et envoie périodiquement des signaux de vie (*heartbeats*).

Avantages

- **Découverte dynamique** : les adresses des services ne sont plus codées en dur, ce qui autorise leur déplacement et leur multiplication transparente.
- **Résilience** : Eureka intègre des mécanismes de tolérance aux pannes, comme le mode d'auto-préservation qui évite de radier abusivement des services en cas de défaillance réseau temporaire.
- **Équilibrage de charge** : associé à la passerelle, Eureka permet de répartir les requêtes entre plusieurs instances d'un même service.

Alternatives et justification du choix

Des alternatives telles que Consul (HashiCorp) ou etcd offrent des fonctionnalités de découverte comparables, voire enrichies (gestion de configuration, contrôles de santé avancés). Néanmoins, Eureka a été privilégié pour son intégration transparente avec Spring Cloud, qui réduit la complexité de mise en œuvre, et pour sa simplicité d'exploitation dans le cadre d'un projet dont l'objectif n'est pas d'introduire une infrastructure de découverte tierce.

3.3.4 Keycloak

Présentation

Keycloak est une solution de gestion des identités et des accès (*Identity and Access Management*) à code source ouvert. Elle implémente les protocoles standards OAuth2 et OpenID Connect, et prend en charge l'authentification, l'autorisation, la gestion des utilisateurs, des rôles et des sessions.

Avantages

- **Standardisation** : en s'appuyant sur des protocoles éprouvés, Keycloak garantit une sécurité conforme à l'état de l'art.
- **Externalisation de la sécurité** : déléguer l'authentification à Keycloak évite de réimplémenter des mécanismes sensibles et sujets aux erreurs.

- **Fonctionnalités avancées** : authentification unique (*SSO*), fédération d'identités, authentification à plusieurs facteurs, gestion fine des rôles.
- **Console d'administration** : une interface d'administration complète facilite la gestion des utilisateurs et des configurations.

Alternatives et justification du choix

Les principales alternatives sont des services managés dans le nuage, tels qu'Auth0 ou Amazon Cognito. Bien que puissants, ces services impliquent une dépendance à un fournisseur externe et un modèle économique fondé sur l'usage. Une autre option aurait consisté à implémenter une sécurité maison à l'aide de Spring Security seul ; cette voie a été écartée car elle aurait exigé de développer et de maintenir des mécanismes de sécurité complexes, avec un risque accru de vulnérabilités. Keycloak offre le meilleur compromis : une solution complète, standardisée, gratuite et auto-hébergée, garantissant la maîtrise totale des données d'identité.

3.3.5 Apache Kafka

Présentation

Apache Kafka est une plateforme distribuée de diffusion d'événements (*event streaming*). Conçue initialement par LinkedIn, elle est devenue la référence pour la construction de systèmes orientés événements et le traitement de flux de données à grande échelle.

Avantages

- **Découplage** : les producteurs et les consommateurs sont totalement indépendants, ce qui favorise l'évolutivité de l'architecture.
- **Durabilité** : les messages sont persistés sur disque et conservés selon une politique de rétention, autorisant la relecture et la résilience.
- **Haut débit** : Kafka est conçu pour traiter de très grands volumes de messages avec une latence faible.
- **Scalabilité horizontale** : le partitionnement des sujets permet de répartir la charge sur plusieurs nœuds et consommateurs.

Alternatives et justification du choix

Les systèmes de messagerie traditionnels, tels que RabbitMQ ou ActiveMQ, constituent des alternatives crédibles. Ils excellent dans les scénarios de file d'attente où chaque message est consommé une seule fois puis supprimé. Toutefois, ils diffèrent fondamentalement de Kafka sur un point : la persistance. Là où une file classique supprime le message

après consommation, Kafka le conserve, ce qui autorise plusieurs consommateurs indépendants et la relecture des événements. Cette propriété est déterminante pour l’extensibilité de notre système : de nouveaux services (notifications, audit, analyse) pourront s’abonner au flux de transactions sans perturber l’existant. C’est cette capacité de rediffusion et ce modèle de journal d’événements durable qui ont motivé le choix de Kafka.

3.3.6 PostgreSQL

Présentation

PostgreSQL est un système de gestion de base de données relationnelle et objet, à code source ouvert, réputé pour sa robustesse, sa conformité aux standards SQL et sa richesse fonctionnelle.

Avantages

- **Fiabilité et intégrité** : PostgreSQL garantit les propriétés ACID, essentielles pour des données financières.
- **Richesse fonctionnelle** : prise en charge des types avancés (JSON, tableaux), des transactions complexes, et d’extensions puissantes.
- **Extensibilité** : l’extension pgvector, notamment, permet la recherche vectorielle, ouvrant des perspectives pour la persistance de la base de connaissances de l’agent.
- **Gratuité et communauté** : solution libre, mature et largement adoptée.

Alternatives et justification du choix

MySQL constitue l’alternative relationnelle la plus répandue ; les bases NoSQL comme MongoDB représentent une autre voie. MySQL, bien que performant, offre historiquement une conformité aux standards et une richesse fonctionnelle moindres que PostgreSQL. Les bases NoSQL, quant à elles, sacrifient l’intégrité relationnelle au profit de la souplesse, ce qui ne convient pas à la nature structurée et sensible de nos données financières. PostgreSQL s’est donc imposé, d’autant que son extension vectorielle offre une trajectoire d’évolution cohérente pour la composante d’intelligence artificielle.

3.3.7 LangChain4j

Présentation

LangChain4j est une bibliothèque Java inspirée de l’écosystème LangChain, dédiée à la construction d’applications exploitant les grands modèles de langage. Elle fournit des abstractions pour l’appel d’outils, la gestion de la mémoire, l’intégration de bases de connaissances et l’orchestration d’agents.

Avantages

- **Orientation agent** : LangChain4j offre un mécanisme mature d'appel d'outils, où le modèle décide de manière autonome des fonctions à invoquer.
- **Abstractions de haut niveau** : la définition d'un service d'IA se fait de manière déclarative, la bibliothèque générant l'implémentation sous-jacente.
- **Support du RAG** : l'intégration de la génération augmentée par récupération est directement prise en charge.
- **Indépendance vis-à-vis du fournisseur** : LangChain4j s'interface avec de multiples fournisseurs de modèles.

Alternatives et justification du choix

La principale alternative dans l'écosystème Java est Spring AI, que nous employons par ailleurs pour l'assistant conversationnel. Le choix de LangChain4j pour l'agent principal se justifie par la maturité supérieure de son mécanisme d'appel d'outils et d'orchestration d'agents, qui constitue précisément le cœur du comportement recherché. Développer une intégration directe avec l'interface d'un fournisseur de modèle, sans bibliothèque intermédiaire, aurait été une autre option ; elle a été écartée car elle aurait imposé de réimplémenter manuellement la boucle de raisonnement, la gestion des outils et la mémoire — fonctionnalités que LangChain4j fournit de manière éprouvée.

3.3.8 Spring AI

Présentation

Spring AI est le projet officiel de l'écosystème Spring pour l'intégration de l'intelligence artificielle générative. Il propose une abstraction unifiée pour dialoguer avec les modèles de langage, gérer la mémoire conversationnelle et implémenter des scénarios de RAG.

Avantages

- **Intégration Spring native** : Spring AI s'utilise comme n'importe quel composant Spring, avec injection de dépendances et auto-configuration.
- **Simplicité de la mémoire** : la gestion de la mémoire conversationnelle est particulièrement fluide, au moyen de conseillers (*advisors*) enfichables.
- **Cohérence** : pour une équipe maîtrisant Spring, la courbe d'apprentissage est minimale.

Alternatives et justification du choix

Spring AI et LangChain4j répondent à des besoins voisins. Plutôt que de les opposer, nous avons choisi de les combiner, chacun étant mobilisé là où il excelle : Spring AI pour l’assistant conversationnel à mémoire, dont l’intégration Spring est particulièrement élégante, et LangChain4j pour l’agent à outils. Cette approche pragmatique illustre une compréhension fine des forces respectives de chaque bibliothèque.

3.3.9 Angular

Présentation

Angular est un cadre de développement d’applications web monopages (*Single Page Applications*), développé et maintenu par Google. Il repose sur TypeScript et adopte une architecture fondée sur des composants.

Avantages

- **Framework complet** : Angular fournit nativement le routage, la gestion des formulaires, les requêtes HTTP et l’injection de dépendances, formant un ensemble cohérent.
- **Typage statique** : l’usage de TypeScript renforce la fiabilité et la maintenabilité du code.
- **Réactivité moderne** : le système de signaux (*signals*) offre une gestion de l’état performante et une détection de changement optimisée.
- **Structure imposée** : Angular impose une organisation claire, bénéfique pour les projets d’envergure et le travail en équipe.

Alternatives et justification du choix

Les principales alternatives sont React et Vue.js. React, très populaire, offre une grande souplesse mais délègue de nombreux choix structurants à des bibliothèques tierces (routage, gestion d’état), ce qui peut nuire à la cohérence. Vue.js propose un bon compromis mais dispose d’un écosystème d’entreprise moins établi. Angular a été retenu pour son caractère complet et structurant, son typage statique natif et son adéquation avec les applications d’entreprise robustes, ainsi que pour la cohérence conceptuelle avec le backend fortement typé.

3.3.10 Docker

Présentation

Docker est une plateforme de conteneurisation qui permet d’empaqueter une application et ses dépendances dans une unité isolée et portable appelée conteneur.

Avantages

- **Portabilité** : un conteneur s’exécute de manière identique sur tout environnement disposant de Docker.
- **Isolation** : chaque composant s’exécute dans son propre conteneur, sans interférence.
- **Reproductibilité** : l’environnement est décrit de manière déclarative, garantissant sa reconstitution à l’identique.
- **Orchestration simplifiée** : Docker Compose permet de démarrer l’ensemble des composants d’infrastructure par une commande unique.

Alternatives et justification du choix

L’installation manuelle de chaque composant sur la machine hôte constituait l’alternative la plus élémentaire ; elle a été écartée en raison de sa lourdeur et de son manque de reproductibilité. Des plateformes d’orchestration plus avancées, comme Kubernetes, auraient pu être envisagées ; toutefois, leur complexité dépasse les besoins d’un environnement de développement et de démonstration. Docker, associé à Docker Compose, offre le juste équilibre entre simplicité, portabilité et reproductibilité.

3.4 Approfondissement des concepts architecturaux

Au-delà du choix des technologies, l’architecture de SmartWealth AI mobilise un ensemble de concepts et de patrons de conception dont la compréhension est essentielle. Cette section les explicite.

3.4.1 Le patron API Gateway

Le patron *API Gateway* consiste à interposer un point d’entrée unique entre les clients et l’ensemble des microservices. Ce patron répond à plusieurs préoccupations. Sans passerelle, chaque client devrait connaître l’adresse de chaque service et gérer lui-même l’authentification auprès de chacun, ce qui multiplierait la complexité côté client et exposerait davantage la surface d’attaque du système. La passerelle centralise ces préoccupations transversales : elle authentifie les requêtes, les route vers le service adéquat, et applique des politiques communes telles que la gestion du partage des ressources entre origines (CORS). Elle offre ainsi au client une interface unifiée, tout en masquant la complexité interne de l’architecture.

3.4.2 Le patron Service Discovery

Dans une architecture distribuée dynamique, les instances de services apparaissent, disparaissent et se déplacent. Coder en dur leurs adresses réseau serait à la fois rigide et fragile. Le patron *Service Discovery* résout ce problème en introduisant un annuaire central. On distingue la découverte côté client, où le client interroge l'annuaire puis sélectionne une instance, de la découverte côté serveur, où un intermédiaire s'en charge. Dans notre architecture, l'annuaire Eureka, associé à la passerelle, met en œuvre ce patron de manière transparente.

3.4.3 Le principe de responsabilité unique appliqué aux services

Chaque microservice de la plateforme est conçu autour d'un unique domaine métier : les utilisateurs, les portefeuilles, les transactions, ou l'intelligence artificielle. Cette application du principe de responsabilité unique à l'échelle des services garantit une forte cohésion interne et un faible couplage externe. Un service peut ainsi évoluer, être testé et déployé indépendamment, sans effet de bord sur les autres.

3.4.4 La cohérence à terme (Eventual Consistency)

L'adoption du principe *database per service* et de la communication asynchrone a une conséquence majeure sur la gestion de la cohérence des données. Dans un système monolithique, une transaction de base de données garantit que toutes les modifications sont appliquées de manière atomique et immédiatement cohérente. Dans notre architecture distribuée, une telle garantie n'est plus possible entre services. Nous adoptons donc le modèle de cohérence à terme : après une transaction, le système atteint un état cohérent au bout d'un délai — le temps que l'événement soit propagé et consommé. Concrètement, lorsqu'un utilisateur effectue un dépôt, la transaction est d'abord enregistrée, puis le solde du portefeuille est mis à jour de manière asynchrone. Durant un très court instant, les deux services peuvent présenter des états transitoires différents, avant de converger. Ce compromis, inhérent aux architectures distribuées, est accepté en connaissance de cause.

3.4.5 La sécurité en profondeur

La sécurité de la plateforme ne repose pas sur une unique barrière, mais sur une défense à plusieurs niveaux, conformément au principe de défense en profondeur.

L'authentification centralisée

Le premier niveau est l'authentification, déléguée à Keycloak. Aucun service ne gère lui-même les mots de passe ; cette responsabilité sensible est confiée à un composant

spécialisé et éprouvé.

La validation des jetons à la passerelle

Le deuxième niveau est la validation systématique des jetons JWT à la passerelle. Toute requête destinée à une ressource protégée doit présenter un jeton valide, dont la signature et l'émetteur sont vérifiés. Une requête sans jeton valide est rejetée avant même d'atteindre les services métier.

La validation au niveau des services

Le troisième niveau est la validation au sein de chaque service. Les services de portefeuille et de transactions sont eux-mêmes configurés en serveurs de ressources : ils revérifient le jeton et en extraient l'identité de l'utilisateur, garantissant qu'un service ne traite que des requêtes légitimes, même dans l'hypothèse d'un contournement de la passerelle.

La propagation de l'identité

Enfin, l'identité de l'utilisateur est propagée de bout en bout. Lorsque l'agent d'intelligence artificielle accède au portefeuille, il transmet le jeton de l'utilisateur, si bien que le service de portefeuille n'expose que les données de cet utilisateur précis. Ce mécanisme prévient toute fuite de données entre utilisateurs.

Synthèse : l'articulation de ces quatre niveaux — authentification centralisée, validation à la passerelle, validation aux services, propagation de l'identité — constitue une défense en profondeur robuste, adaptée à la sensibilité des données financières manipulées.

3.5 Conception détaillée

3.5.1 Diagramme d'architecture globale

Le diagramme d'architecture globale représente l'organisation d'ensemble du système : le frontend, la passerelle, les composants d'infrastructure, les microservices, leurs bases de données et les services externes, ainsi que les flux de communication qui les relient.

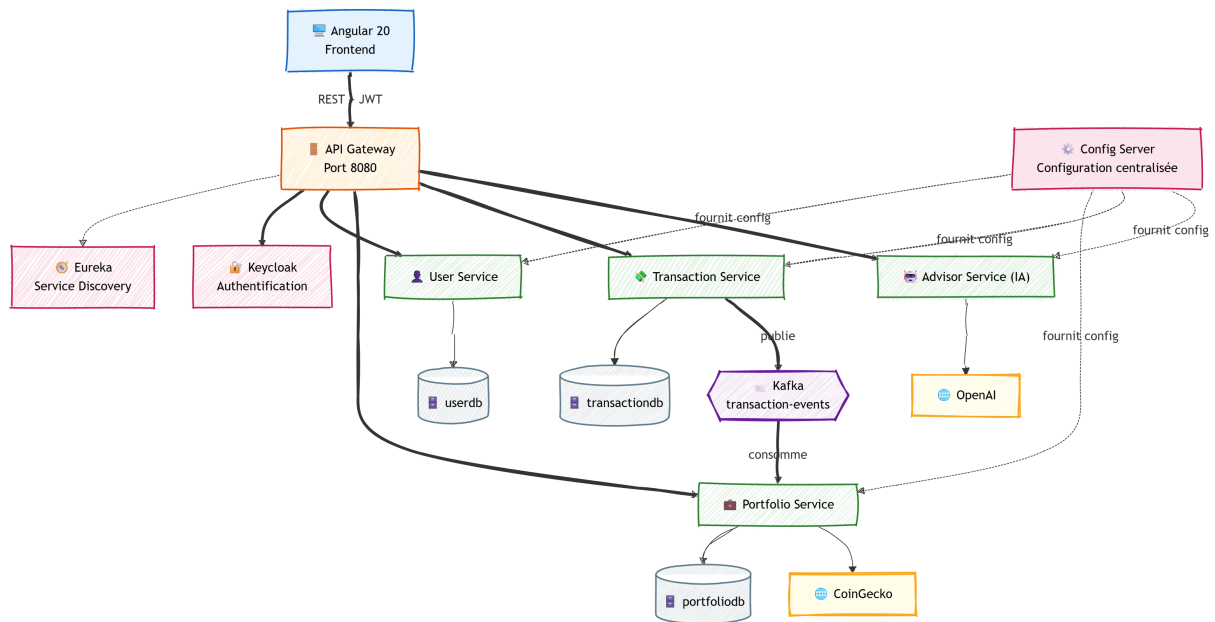


FIGURE 3.2 – Architecture technique globale de SmartWealth AI

3.5.2 Diagramme de classes

Le diagramme de classes modélise les entités métier et leurs relations. Il constitue la traduction, en termes de conception orientée objet, du domaine fonctionnel identifié au chapitre précédent.

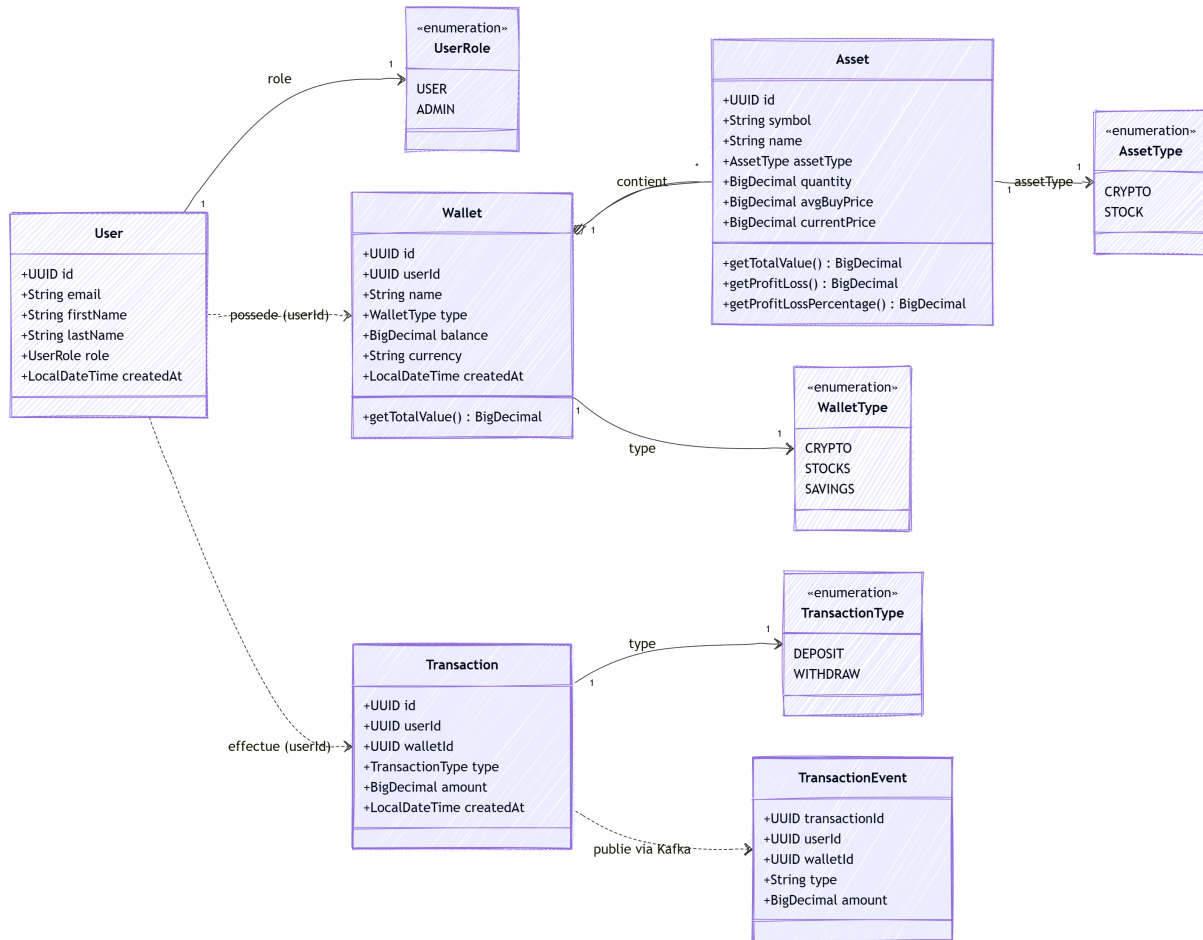


FIGURE 3.3 – Diagramme de classes

Les entités principales sont les suivantes :

- **User** : représente un utilisateur de la plateforme, caractérisé par son identifiant, son courriel, son nom, son prénom et son rôle.
- **Wallet** : représente un portefeuille, doté d'un nom, d'un type, d'une devise et d'un solde de liquidités. Il agrège un ensemble d'actifs.
- **Asset** : représente un actif détenu au sein d'un portefeuille, caractérisé par son symbole, sa quantité, son prix moyen d'achat et son prix actuel. Il expose des méthodes de calcul de valorisation et de performance.
- **Transaction** : représente un mouvement financier (dépôt ou retrait) affectant un portefeuille.
- **TransactionEvent** : représente l'événement publié sur le bus de messagerie à la suite d'une transaction.

Note de conception sur les relations inter-services : les relations entre les entités appartenant à des services distincts (par exemple entre **User** et **Wallet**) ne sont pas matérialisées par des clés étrangères, mais par de simples identifiants (UUID). Cela résulte directement du principe *database per service* : chaque entité réside dans la base de son service propriétaire, et les liens inter-services sont logiques plutôt que physiques.

3.5.3 Diagrammes de séquence

Les diagrammes de séquence décrivent les interactions entre les composants dans le temps, en mettant en évidence l'ordre chronologique des échanges de messages.

Séquence : Authentification

Ce diagramme illustre le flux de connexion. Le frontend transmet les identifiants à la passerelle, qui les achemine vers le service utilisateur. Ce dernier sollicite Keycloak, obtient les jetons et les retourne au client. Les requêtes ultérieures sont validées à chaque passage par la passerelle.

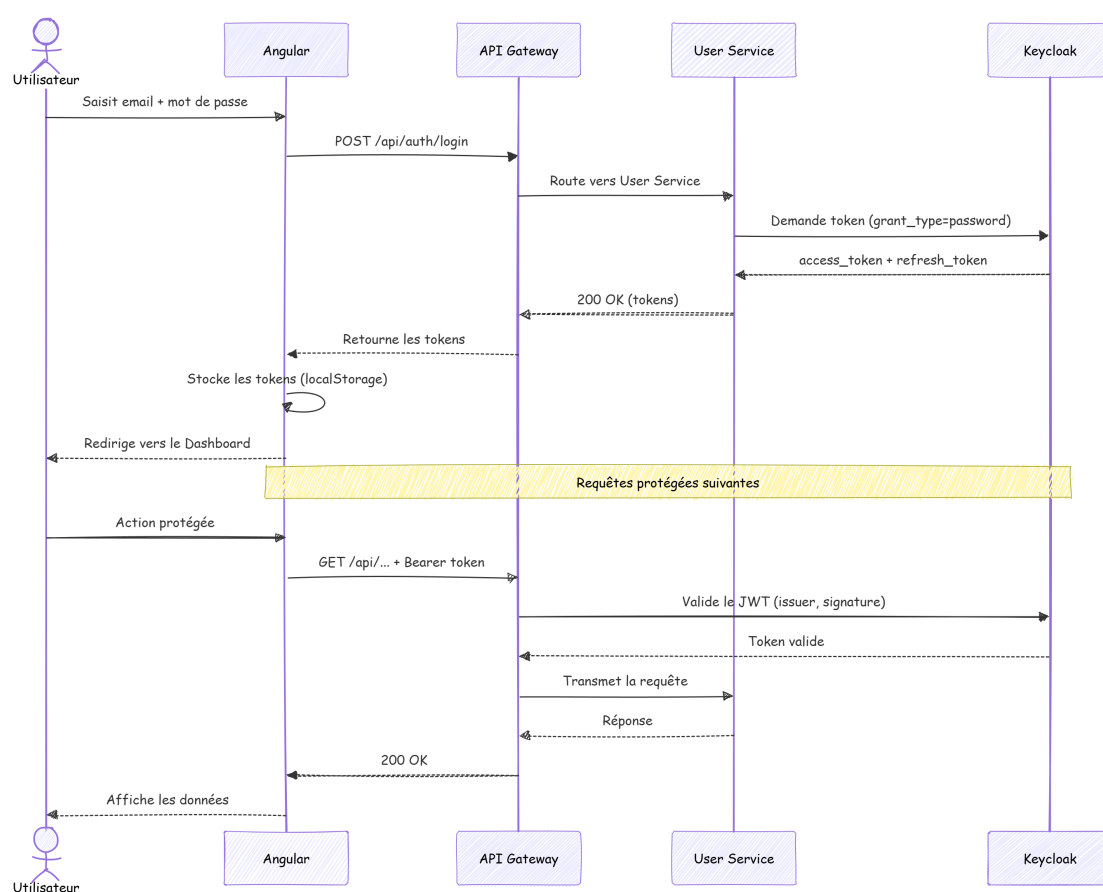


FIGURE 3.4 – Diagramme de séquence — Authentification

Séquence : Transaction et propagation événementielle

Ce diagramme met en scène le traitement d'une transaction. Le service de transactions vérifie d'abord le solde de manière synchrone, enregistre la transaction, puis publie un événement. Le service de portefeuille consomme cet événement et met à jour le solde. Le diagramme distingue clairement la partie synchrone de la partie asynchrone.

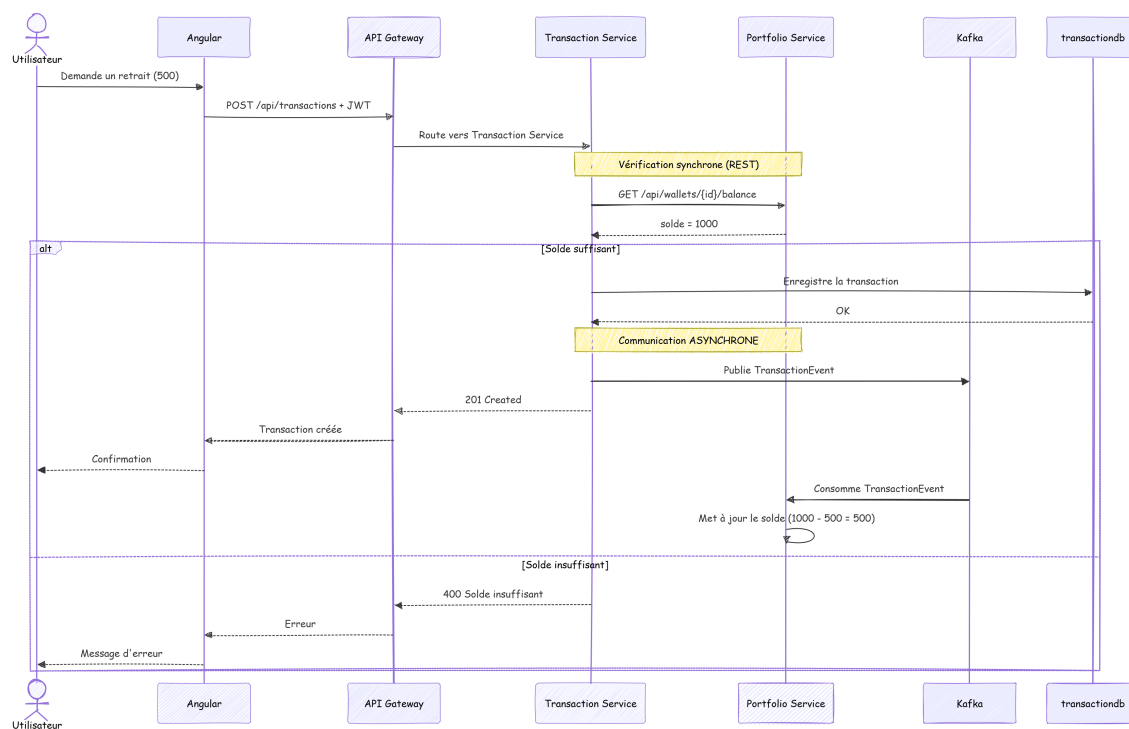


FIGURE 3.5 – Diagramme de séquence — Transaction (Kafka)

Séquence : Raisonnement de l'agent IA

Ce diagramme représente le raisonnement de l'agent. À réception d'une question, l'agent décide d'invoquer un ou plusieurs outils pour accéder aux données réelles, consulte la base de connaissances, puis synthétise une réponse. La possibilité d'enchaîner plusieurs outils illustre le raisonnement multi-étapes.

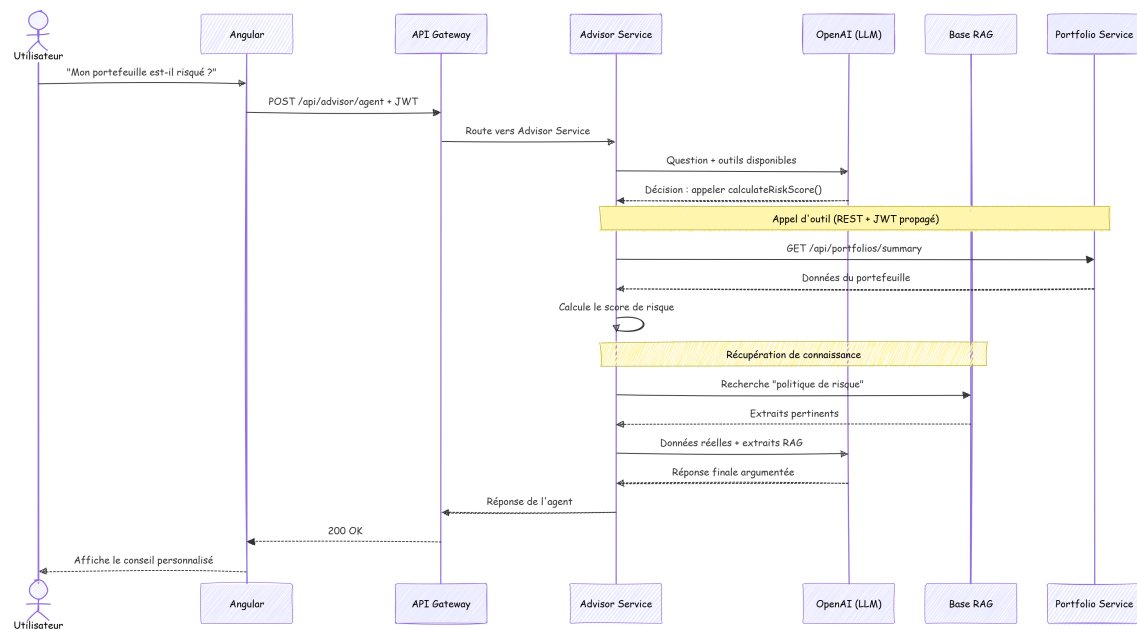


FIGURE 3.6 – Diagramme de séquence — Agent IA

3.5.4 Diagrammes d'activité

Les diagrammes d'activité modélisent le déroulement des processus métier, en explicitant les décisions et les branches alternatives.

Activité : Traitement d'une transaction

Ce diagramme détaille le cheminement d'une transaction, depuis la saisie jusqu'à la mise à jour du solde, en passant par la décision relative au type de transaction et la vérification du solde pour les retraits.

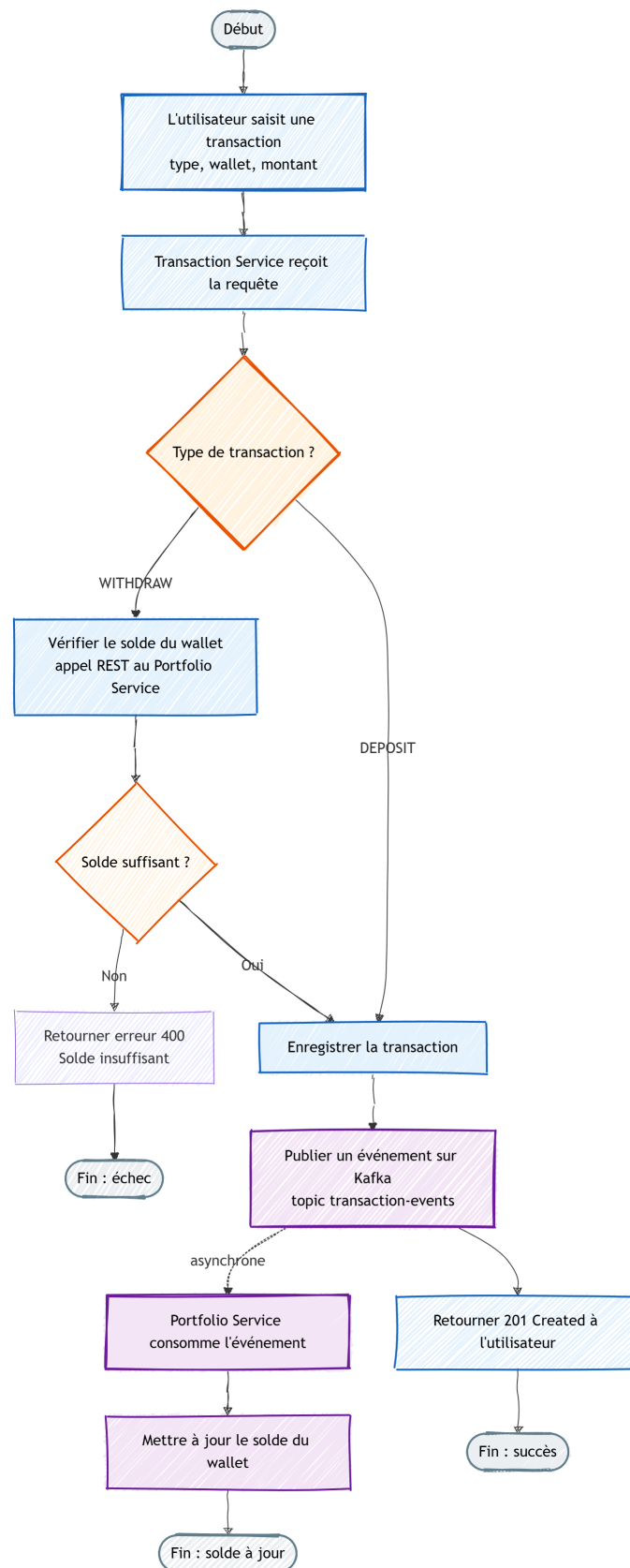


FIGURE 3.7 – Diagramme d'activité — Traitement d'une transaction

Activité : Raisonnement de l'agent

Ce diagramme décrit la logique décisionnelle de l'agent : évaluation du besoin d'un outil, appel éventuel, consultation de la base de connaissances, et boucle de raisonnement jusqu'à la formulation de la réponse.

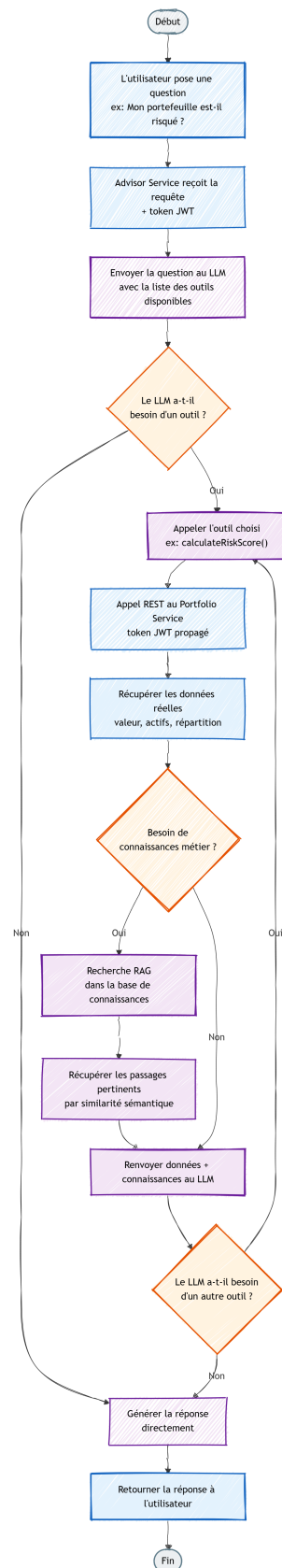


FIGURE 3.8 – Diagramme d'activité — Raisonnement de l'agent

Conclusion

Ce chapitre a exposé et justifié les choix architecturaux et technologiques du projet, puis a détaillé la conception du système à travers les principaux diagrammes UML. Ces fondations de conception étant posées, le chapitre suivant présente la réalisation concrète de la plateforme, module par module.

Chapitre 4

Implémentation de la solution

Introduction

Ce chapitre présente la concrétisation des choix de conception exposés précédemment. La réalisation est décrite module par module, en suivant l'ordre logique de construction du système. Pour chaque module, nous exposons les principes de fonctionnement, les points techniques saillants, des extraits de code représentatifs et les résultats obtenus, illustrés par des captures d'écran de l'application.

4.1 Environnement de développement et de déploiement

4.1.1 Outils de développement

Le développement s'est appuyé sur un ensemble d'outils standards de l'ingénierie logicielle. L'environnement de développement intégré IntelliJ IDEA a été utilisé pour le backend Java, tandis que l'outil Postman a servi à tester les interfaces REST. Le système de gestion de versions Git a assuré le suivi des évolutions du code.

4.1.2 Conteneurisation avec Docker

L'ensemble des composants d'infrastructure — bases de données PostgreSQL, Keycloak, Apache Kafka — a été conteneurisé au moyen de Docker et orchestré par Docker Compose. Cette approche garantit un environnement homogène et reproductible, et simplifie considérablement le démarrage de la plateforme.

```

services:
  postgres-user:
    image: postgres:16-alpine
    container_name: postgres-user
    restart: unless-stopped
    environment:
      POSTGRES_DB: userdb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5432:5432"
    volumes:
      - user-db-data:/var/lib/postgresql/data
    networks:
      - smartwealth-net
  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: pgadmin
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@smart.com
      PGADMIN_DEFAULT_PASSWORD: admin123
    ports:
      - "5050:80"
    networks:
      - smartwealth-net
    depends_on:
      - postgres-user

  postgres-portfolio:
    image: postgres:16-alpine
    container_name: postgres-portfolio
    restart: unless-stopped
    environment:
      POSTGRES_DB: portfolio
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5433:5432" # 5433 on your PC (5432 is taken by user DB)
    volumes:
      - portfolio-db-data:/var/lib/postgresql/data
    networks:
      - smartwealth-net
  postgres-transaction:
    image: postgres:16-alpine
    container_name: postgres-transaction
    restart: unless-stopped
    environment:
      POSTGRES_DB: transactiondb
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5434:5432"
    volumes:
      - transaction-db-data:/var/lib/postgresql/data
    networks:
      - smartwealth-net

  postgres-keycloak:
    image: postgres:16-alpine
    container_name: postgres-keycloak
    restart: unless-stopped
    environment:
      POSTGRES_DB: keycloakdb
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: keycloak123
    volumes:
      - keycloak-db-data:/var/lib/postgresql/data
    networks:
      - smartwealth-net
  keycloak:
    image: quay.io/keycloak/keycloak:25.0
    container_name: keycloak
    restart: unless-stopped
    command: start-dev
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://postgres-keycloak:5432/keycloakdb
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: keycloak123
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin123
      KC_HOSTNAME: localhost
      KC_HOSTNAME_STRICT: "false"
      KC_HTTP_ENABLED: "true"
    ports:
      - "8181:8080"
    depends_on:
      - postgres-keycloak
    networks:
      - smartwealth-net
  kafka:
    image: confluentinc/cp-kafka:7.6.1
    container_name: kafka
    restart: unless-stopped
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:29093
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:29092,CONTROLLER://0.0.0.0:29093,EXTERNAL://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      CLUSTER_ID: MKU30EVbNTcWNTJENDM2Qk
    networks:
      - smartwealth-net
  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    container_name: kafka-ui
    restart: unless-stopped
    ports:
      - "8090:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: smartwealth
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
    depends_on:
      - kafka
    networks:
      - smartwealth-net

volumes:
  user-db-data:
  keycloak-db-data:
  portfolio-db-data:
  transaction-db-data:
networks:
  smartwealth-net:
    driver: bridge

```

FIGURE 4.1 – Extrait du fichier docker-compose.yml

4.2 Module 1 : Authentification et sécurité

4.2.1 Principe de fonctionnement

Le module d'authentification constitue la porte d'entrée de la plateforme. Il repose sur l'intégration de Keycloak, qui centralise la gestion des identités. Le processus d'authentification se déroule comme suit : l'utilisateur soumet ses identifiants, le service utilisateur les échange auprès de Keycloak contre un jeton d'accès (JWT) et un jeton de rafraîchissement, puis ces jetons sont retournés au client qui les conserve.

4.2.2 Le flux d'authentification côté serveur

Le service utilisateur expose un point d'accès dédié à la connexion. Celui-ci délègue à Keycloak la validation des identifiants au moyen du flux OAuth2 par mot de passe. L'extrait suivant illustre la structure de la réponse d'authentification retournée au client, comportant le jeton d'accès, le jeton de rafraîchissement et leur durée de validité.

```
1 public class TokenResponse {  
2     private String accessToken;  
3     private String refreshToken;  
4     private Long   expiresIn;  
5     private String tokenType;  
6 }
```

Listing 4.1 – Structure de la réponse d'authentification

4.2.3 La validation des jetons à la passerelle

La passerelle est configurée en tant que serveur de ressources OAuth2. Elle valide le jeton JWT présenté à chaque requête en vérifiant sa signature et son émetteur auprès de Keycloak. Les points d'accès publics — connexion, inscription — sont explicitement exemptés de cette validation, de même que les requêtes de pré-vérification CORS.

```
1 http  
2     .cors(Customizer.withDefaults())  
3     .csrf(ServerHttpSecurity.CsrfSpec::disable)  
4     .authorizeExchange(exchange -> exchange  
5         .pathMatchers(HttpMethod.OPTIONS, "/*").permitAll()  
6         .pathMatchers("/api/auth/*").permitAll()  
7         .pathMatchers("/api/users/register").permitAll()  
8         .anyExchange().authenticated())  
9     .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))  
    ;
```

Listing 4.2 – Configuration de sécurité de la passerelle (extrait)

4.2.4 La gestion des jetons côté client

Côté frontend, un intercepteur HTTP attache automatiquement le jeton d'accès à chaque requête sortante destinée aux points d'accès protégés. Lorsqu'une requête échoue en raison d'un jeton expiré, l'intercepteur déclenche le renouvellement du jeton au moyen du jeton de rafraîchissement, puis réemet la requête initiale de manière transparente pour l'utilisateur.

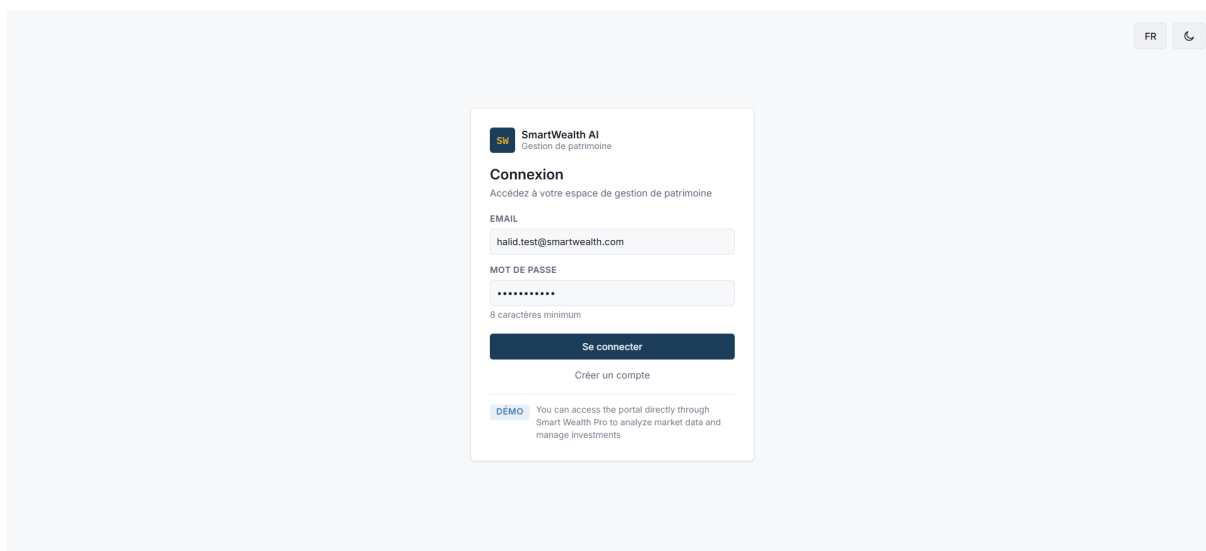


FIGURE 4.2 – Interface de connexion / inscription

4.3 Module 2 : Gestion du portefeuille

4.3.1 Principe de fonctionnement

Le service de portefeuille gère la création des portefeuilles et l'ajout d'actifs. Sa particularité réside dans l'intégration des prix de marché en temps réel et le calcul dynamique de la valorisation et de la performance.

4.3.2 L'intégration des prix de marché

Lors de l'ajout d'un actif, le service interroge l'API CoinGecko afin d'obtenir le prix de marché actuel du symbole concerné. Ce prix est ensuite utilisé pour calculer la valorisation. Afin d'éviter de solliciter l'API externe à chaque lecture, les prix sont mis en cache et actualisés périodiquement.

4.3.3 Les calculs de valorisation et de performance

Les calculs financiers sont encapsulés au sein des entités du domaine, conformément aux principes de la conception orientée objet. La valorisation d'un actif correspond au produit de son prix actuel par la quantité détenue. La plus-value ou moins-value se calcule comme la différence entre le prix actuel et le prix moyen d'achat, multipliée par la quantité.

```
1 public BigDecimal getTotalValue() {  
2     return currentPrice.multiply(quantity);  
3 }  
4  
5 public BigDecimal getProfitLoss() {  
6     return currentPrice.subtract(avgBuyPrice).multiply(quantity);  
7 }  
8  
9 public BigDecimal getProfitLossPercentage() {  
10    if (avgBuyPrice.compareTo(BigDecimal.ZERO) == 0)  
11        return BigDecimal.ZERO;  
12    return currentPrice.subtract(avgBuyPrice)  
13        .divide(avgBuyPrice, 4, RoundingMode.HALF_UP)  
14        .multiply(new BigDecimal("100"));  
15 }
```

Listing 4.3 – Méthodes de calcul de l'entité Asset

Précision sur le prix moyen d'achat : le prix moyen désigne le prix unitaire d'acquisition, et non le montant total investi. Ainsi, l'achat de 0,001 unité pour un montant total de 70 correspond à un prix moyen unitaire de 70 000. Cette distinction est essentielle : une confusion à la saisie fausserait le calcul de la performance, le pourcentage étant obtenu par division par le prix moyen.

4.3.4 La valorisation consolidée

La valorisation d'un portefeuille correspond à la somme des valorisations de ses actifs, sauf pour les portefeuilles d'épargne dont la valeur est constituée des liquidités. La valorisation du patrimoine global agrège l'ensemble des portefeuilles.

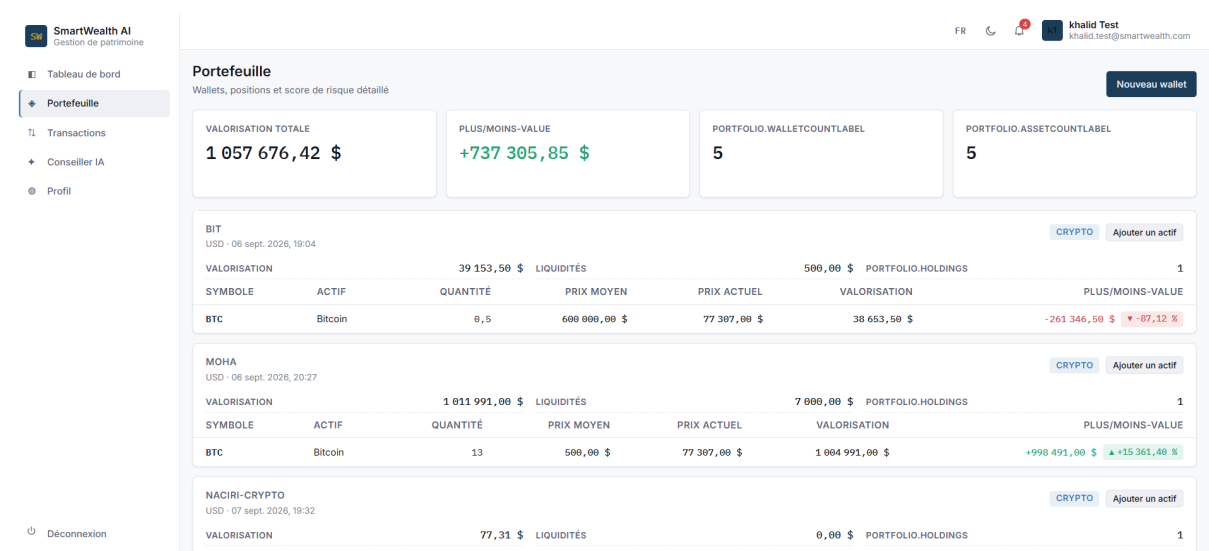


FIGURE 4.3 – Page portefeuille avec valorisation et performance

4.3.5 Interface de création d'un portefeuille

La création d'un portefeuille s'effectue au moyen d'un formulaire dédié, dans lequel l'utilisateur renseigne le nom du portefeuille, son type (cryptomonnaies, actions ou épargne) et sa devise. À la validation, une requête est émise vers le service de portefeuille, qui persiste le nouveau portefeuille et l'associe à l'utilisateur courant.

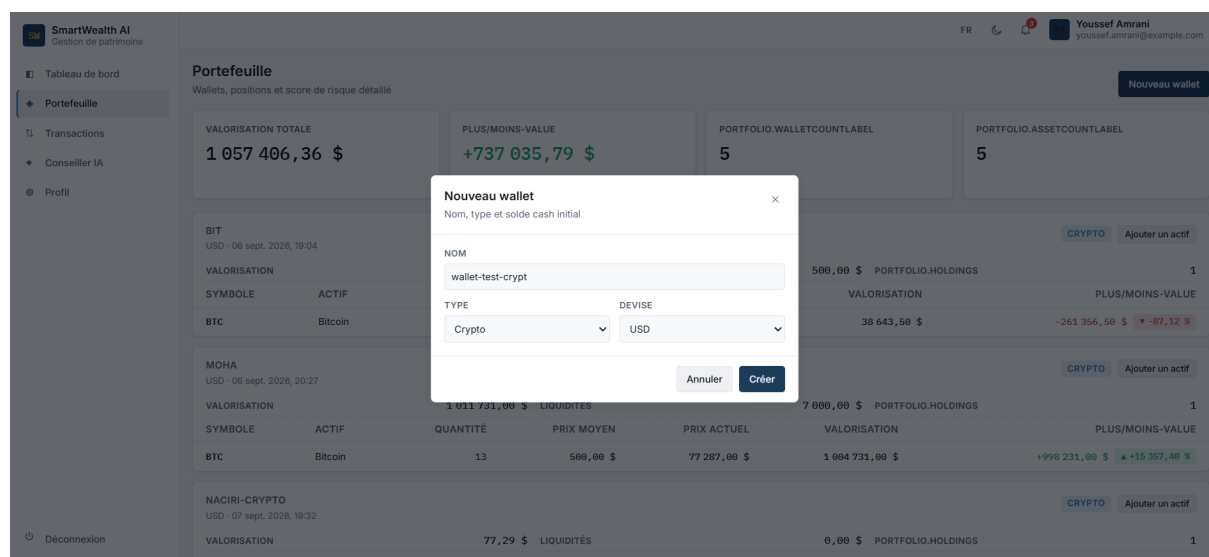


FIGURE 4.4 – Interface d'ajout d'un portefeuille (Add wallet)

4.3.6 Interface d'ajout d'un actif

L'ajout d'un actif à un portefeuille se fait via un formulaire où l'utilisateur saisit le symbole de l'actif, sa quantité et son prix moyen d'achat. Lors de la soumission, le service

de portefeuille interroge l'API CoinGecko pour récupérer le prix de marché actuel, puis calcule la valorisation et la performance de l'actif.

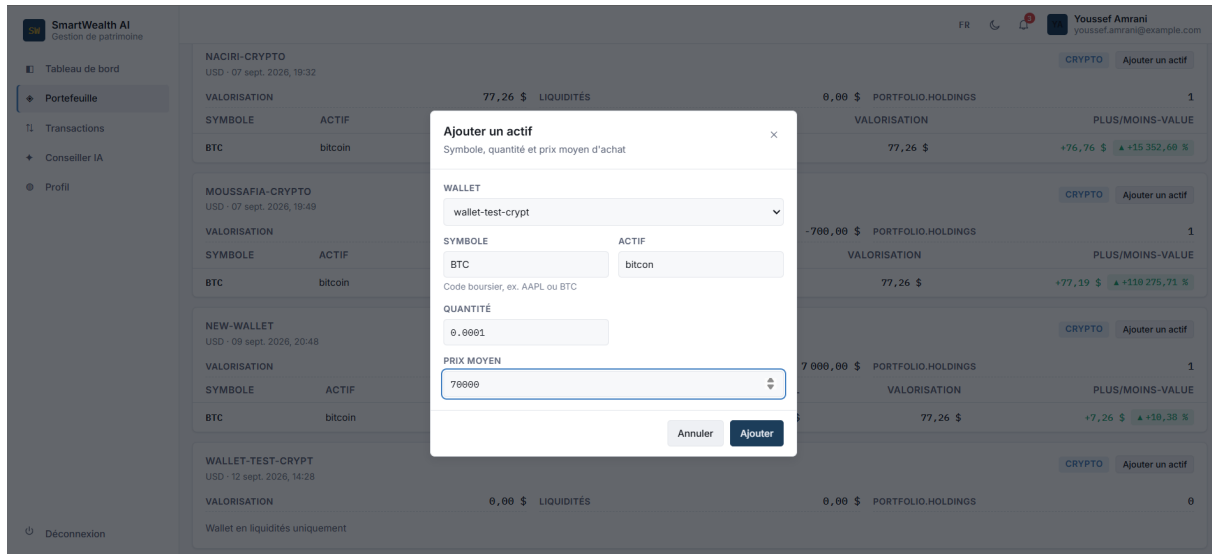


FIGURE 4.5 – Interface d'ajout d'un actif (Add actif)

4.4 Module 3 : Transactions et communication événementielle

4.4.1 Principe de fonctionnement

Le service de transactions gère les dépôts et les retraits. Il illustre de manière emblématique la combinaison des communications synchrone et asynchrone qui caractérise l'architecture du projet.

4.4.2 La vérification synchrone du solde

Avant d'autoriser un retrait, le service de transactions doit s'assurer que le solde du portefeuille est suffisant. Cette information appartenant au service de portefeuille, une communication synchrone est établie : le service de transactions interroge, via une requête REST, le solde du portefeuille concerné, en propageant le jeton de l'utilisateur.

```

1  if (request.getType() == TransactionType.WITHDRAW) {
2      BigDecimal balance = fetchWalletBalance(
3          request.getWalletId(), bearerToken);
4      if (balance.compareTo(request.getAmount()) < 0) {
5          throw new ResponseStatusException(
6              HttpStatus.BAD_REQUEST, "Solde insuffisant");
7      }

```

```
8 }
```

Listing 4.4 – Vérification synchrone du solde avant retrait

4.4.3 La publication de l'événement

Une fois la transaction validée et enregistrée, le service publie un événement sur le sujet Kafka `transaction-events`. L'événement est structuré et sérialisé au format JSON. Il est notable que le service de transactions ne fait aucune référence directe au service de portefeuille : il se contente d'annoncer l'événement.

```
1 producer.publish(TransactionEvent.builder()  
2     .transactionId(saved.getId())  
3     .userId(saved.getUserId())  
4     .walletId(saved.getWalletId())  
5     .type(saved.getType().name())  
6     .amount(saved.getAmount())  
7     .build());
```

Listing 4.5 – Publication de l'événement de transaction

4.4.4 La consommation de l'événement

Le service de portefeuille est abonné au sujet `transaction-events`. À la réception d'un événement, il met à jour le solde du portefeuille concerné : un dépôt l'augmente, un retrait le diminue. Ce traitement s'effectue de manière totalement asynchrone, indépendamment de la réponse retournée à l'utilisateur.

```
1 @KafkaListener(topics = "transaction-events",  
2               groupId = "portfolio-service")  
3 public void onTransaction(TransactionEvent event) {  
4     Wallet wallet = walletRepository  
5         .findById(event.getWalletId()).orElseThrow();  
6     switch (event.getType()) {  
7         case "DEPOSIT" -> wallet.setBalance(  
8             wallet.getBalance().add(event.getAmount()));  
9         case "WITHDRAW" -> wallet.setBalance(  
10             wallet.getBalance().subtract(event.getAmount()));  
11     }  
12     walletRepository.save(wallet);  
13 }
```

Listing 4.6 – Consommation de l'événement côté portefeuille

Compromis assumé : la vérification synchrone du solde crée un couplage temporaire entre le service de transactions et celui de portefeuille. Une alternative pleinement découplée consisterait à recourir au patron *Saga*, dans lequel le service de portefeuille validerait le retrait et publierait, en cas d'échec, un événement de rejet. Ce patron, plus robuste mais plus complexe, constitue une perspective d'évolution identifiée.

4.4.5 Page principale des transactions

La page des transactions offre à l'utilisateur une vue d'ensemble de l'ensemble de ses mouvements financiers. Elle présente l'historique des transactions, avec pour chacune son type, son montant, le portefeuille concerné et sa date. Cette interface constitue le point d'entrée pour consulter et initier les opérations.

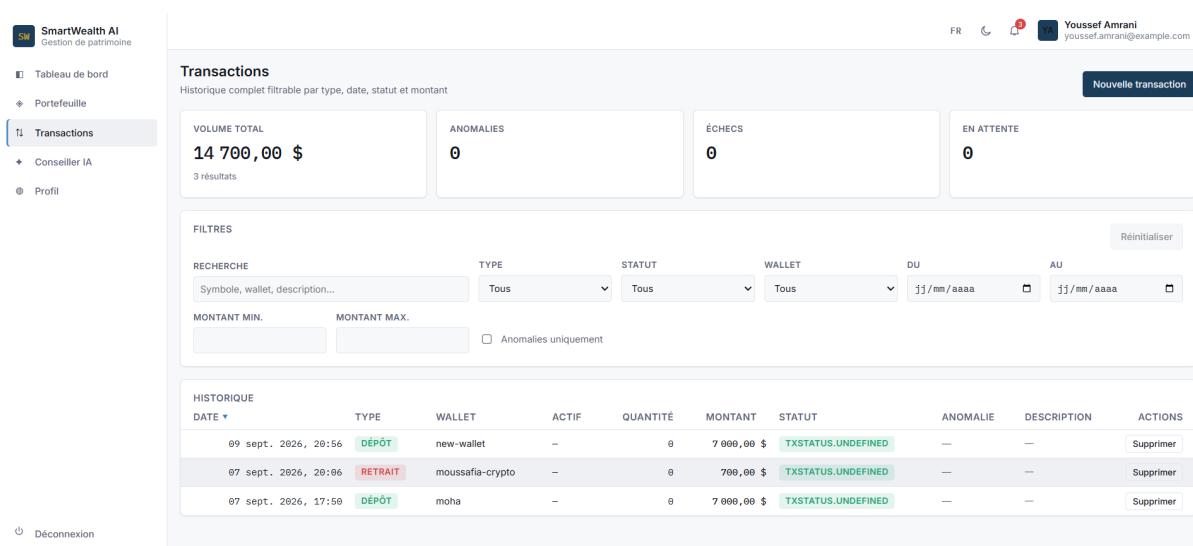


FIGURE 4.6 – Page principale des transactions

4.4.6 Interface d'ajout d'une transaction

La création d'une transaction s'effectue au moyen d'un formulaire dans lequel l'utilisateur sélectionne le portefeuille, choisit le type d'opération (dépôt ou retrait) et saisit le montant. À la validation, le service de transactions applique la logique décrite précédemment : vérification du solde le cas échéant, enregistrement, puis publication de l'événement.

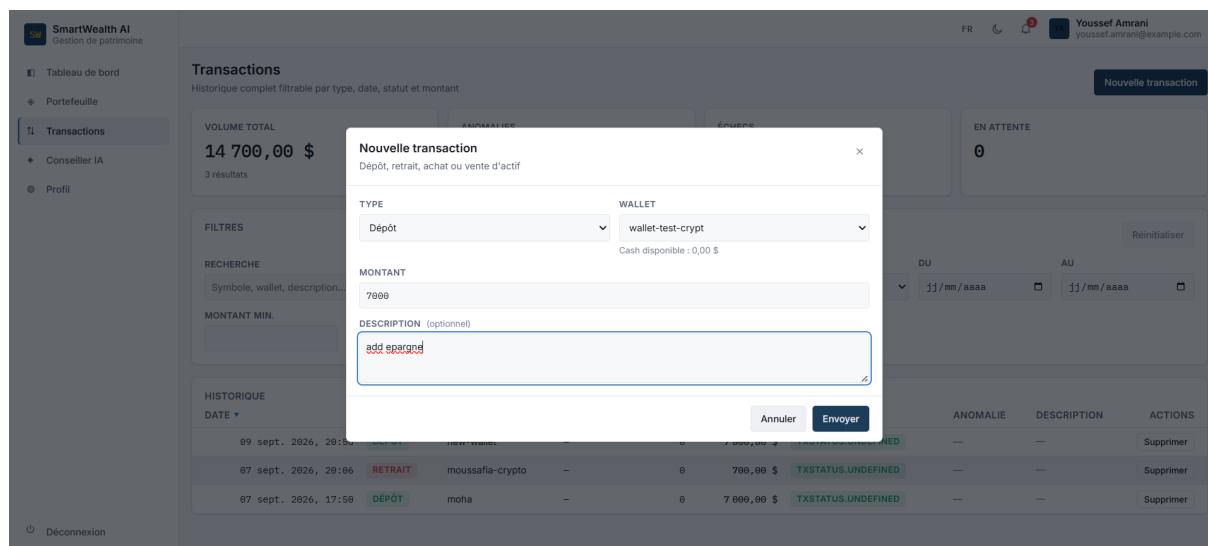


FIGURE 4.7 – Interface d’ajout d’une transaction (Add transaction)

4.4.7 Propagation de l’événement dans Kafka

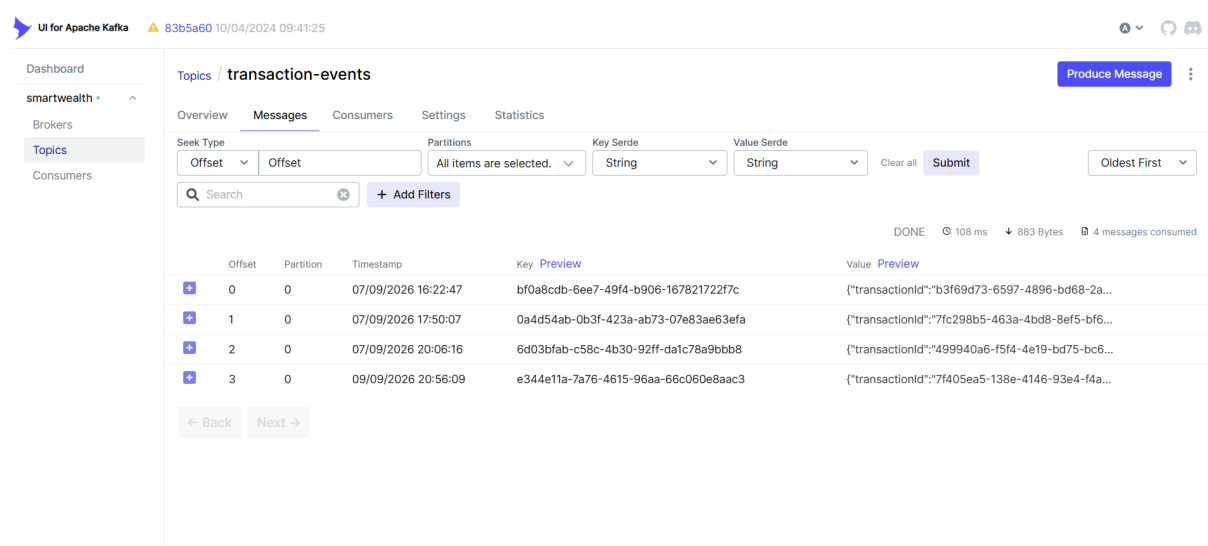


FIGURE 4.8 – Transaction et visualisation de l’événement dans Kafka UI

4.5 Module 4 : Agent d’intelligence artificielle

4.5.1 Principe de fonctionnement

L’agent d’intelligence artificielle constitue l’apport le plus innovant de la plateforme. Il ne se limite pas à générer du texte : il raisonne, invoque des outils pour accéder aux données réelles, mobilise une mémoire conversationnelle et exploite une base de connaissances. Il

combine ainsi trois mécanismes complémentaires, mis en œuvre à l'aide de LangChain4j et de Spring AI.

4.5.2 La conversation avec mémoire

Le premier mécanisme, réalisé avec Spring AI, dote l'assistant d'une mémoire conversationnelle. Au sein d'une session, l'assistant se souvient des échanges précédents, ce qui autorise un dialogue naturel et contextualisé. La mémoire est indexée par identifiant d'utilisateur, et une fonction permet de la réinitialiser.

```
1 @Bean
2 public ChatClient conversationChatClient(
3     ChatClient.Builder builder, ChatMemory chatMemory) {
4     return builder
5         .defaultSystem("Tu es SmartWealth, un assistant financier.")
6         .defaultAdvisors(
7             MessageChatMemoryAdvisor.builder(chatMemory).build())
8         .build();
9 }
```

Listing 4.7 – Configuration de la mémoire conversationnelle (Spring AI)

4.5.3 L'agent à outils

Le deuxième mécanisme, réalisé avec LangChain4j, confère à l'agent la capacité d'invoquer des outils. Un outil est une méthode Java, annotée de manière à être exposée au modèle de langage. Face à une question, le modèle décide de lui-même s'il doit appeler un outil pour obtenir une information. Les outils de SmartWealth AI permettent de récupérer la valeur du portefeuille, le nombre de transactions et de calculer un score de risque.

```
1 @Tool("Récupère la valeur totale du portefeuille en USD")
2 public double getPortfolioValue() {
3     PortfolioSummaryDto summary = fetchSummary();
4     return summary.totalPortfolioValue().doubleValue();
5 }
```

Listing 4.8 – Définition d'un outil accessible à l'agent

Lorsqu'il invoque un tel outil, l'agent déclenche un appel REST vers le service de portefeuille, en propageant le jeton de l'utilisateur, garantissant ainsi qu'il n'accède qu'aux données autorisées.

4.5.4 La base de connaissances (RAG)

Le troisième mécanisme est la génération augmentée par récupération (RAG). Il permet à l'agent de fonder ses réponses sur une base documentaire propre au domaine, plutôt

que sur les seules connaissances générales du modèle.

Le principe se déroule en deux temps. Lors d’une phase d’**ingestion**, réalisée au démarrage, les documents de connaissance sont découpés en fragments, convertis en représentations vectorielles (*embeddings*) et stockés dans un index. Lors d’une phase de **récupération**, à chaque question, celle-ci est également vectorisée, et les fragments dont la représentation est la plus proche sont extraits et fournis au modèle pour formuler sa réponse.

```
1 EmbeddingStoreIngestor ingestor = EmbeddingStoreIngestor.builder()
2   .documentSplitter(DocumentSplitters.recursive(300, 30))
3   .embeddingModel(embeddingModel)
4   .embeddingStore(embeddingStore)
5   .build();
6 ingestor.ingest(documents);
```

Listing 4.9 – Ingestion des documents dans l’index vectoriel

Choix d’un modèle d’embedding local : la vectorisation des documents est réalisée par un modèle exécuté localement, et non par un service distant. Ce choix évite toute dépendance à une interface externe pour l’indexation, supprime les coûts associés et garantit la confidentialité des documents. Seule la génération finale des réponses fait appel au modèle de langage distant.

4.5.5 La synthèse : un agent complet

La force de l’agent réside dans la combinaison de ces trois mécanismes. Face à une question telle que « Mon portefeuille est-il trop risqué et que me conseilles-tu ? », l’agent invoque l’outil de calcul de risque pour obtenir des données réelles, consulte la base de connaissances pour en extraire la politique de risque applicable, et synthétise l’ensemble en une recommandation personnalisée et argumentée. C’est cette orchestration, pilotée par le modèle de langage, qui caractérise un véritable agent.

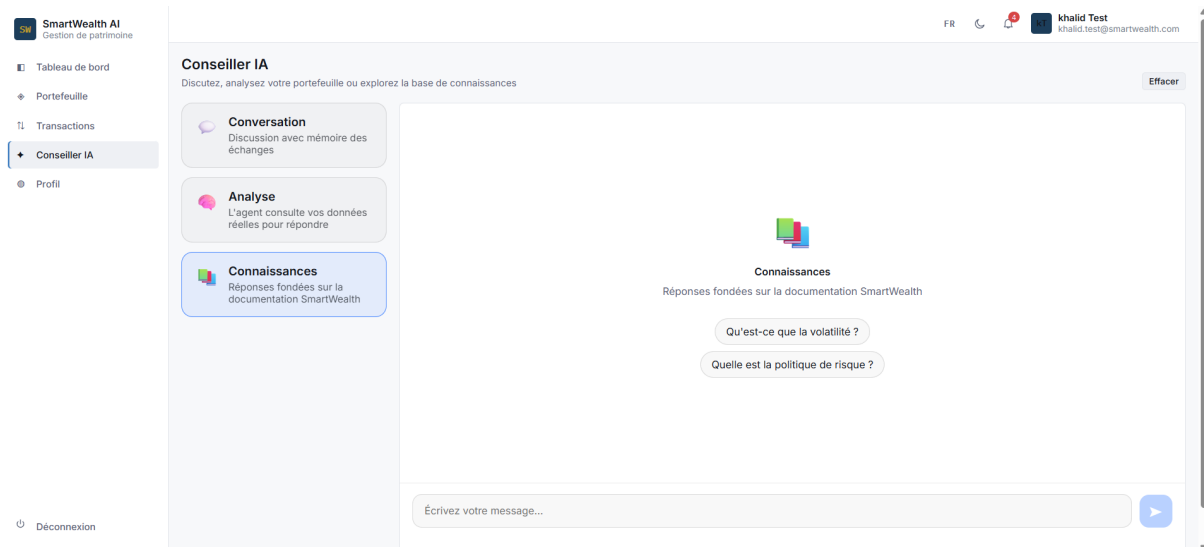


FIGURE 4.9 – Interface conversationnelle de l’agent IA

4.6 Interface utilisateur

L’interface, développée avec Angular, offre une expérience réactive et cohérente. Elle exploite le système de signaux d’Angular pour la gestion de l’état, garantissant une mise à jour efficace de l’affichage. L’interface est multilingue et propose un thème clair et un thème sombre, adaptables selon les préférences de l’utilisateur.



Ce chapitre a détaillé la réalisation concrète des quatre modules de SmartWealth AI, en illustrant les points techniques déterminants par des extraits de code. L'ensemble constitue une plateforme fonctionnelle, sécurisée, événementielle et intelligente, conforme aux besoins exprimés et aux choix de conception établis.

Conclusion générale et perspectives

Le projet SmartWealth AI, objet de ce mémoire, avait pour ambition de concevoir et de réaliser une plateforme de gestion de patrimoine moderne, alliant robustesse architecturale et intelligence artificielle. Au terme de ce travail, il convient d'en dresser le bilan et d'en envisager les prolongements.

Bilan du travail réalisé

Sur le plan architectural, nous avons conçu et mis en œuvre une architecture micro-services complète, articulée autour d'une passerelle applicative, d'un annuaire de services et d'une sécurité centralisée déléguée à Keycloak. Le principe *database per service* a été rigoureusement appliqué, garantissant l'indépendance et le découplage des services. La communication inter-services combine judicieusement les paradigmes synchrone et asynchrone, ce dernier étant assuré par Apache Kafka pour les traitements ne nécessitant pas de réponse immédiate.

Sur le plan de l'intelligence artificielle, nous avons intégré un agent capable de raisonner en plusieurs étapes. Cet agent conjugue trois mécanismes : l'appel d'outils pour accéder aux données réelles de l'utilisateur, une mémoire conversationnelle assurant la continuité du dialogue, et une base de connaissances exploitée par la technique RAG. Cette combinaison élève l'assistant au rang de véritable conseiller contextualisé.

Sur le plan des compétences, ce projet nous a permis d'appliquer concrètement un ensemble de concepts fondamentaux de l'ingénierie logicielle moderne : la conception d'architectures distribuées, la sécurisation des échanges, la messagerie événementielle, l'intégration de l'intelligence artificielle et le développement full-stack. Il a également été l'occasion de nous confronter à des problématiques réelles — gestion de la cohérence distribuée, propagation de l'identité, orchestration de services hétérogènes — et d'y apporter des réponses argumentées.

Limites et difficultés

Comme tout projet, celui-ci comporte des limites que l'honnêteté intellectuelle commande de reconnaître. La vérification synchrone du solde introduit un couplage entre deux services, là où une approche pleinement événementielle serait préférable. Le score de risque repose sur une heuristique volontairement simple, qui ne saurait se substituer

à un modèle financier rigoureux. Enfin, l'index vectoriel de la base de connaissances est maintenu en mémoire, ce qui convient à une base de taille modeste mais devrait être reconsidéré à plus grande échelle. Par ailleurs, le tableau de bord (*dashboard*) n'a pas fait l'objet d'un développement complet : dans l'état actuel, il se limite à un affichage statique et ne reflète pas encore dynamiquement les données réelles de l'utilisateur. Son implémentation aboutie constitue l'une des évolutions prioritaires.

Perspectives d'évolution

Plusieurs axes d'amélioration se dégagent naturellement de ce travail :

- **Tableau de bord dynamique** : le développement complet du tableau de bord, actuellement statique, afin qu'il agrège et visualise en temps réel la valorisation, la performance et la répartition du patrimoine.
- **Cohérence distribuée** : l'adoption du patron *Saga* permettrait de gérer les transactions distribuées de manière pleinement découplée et résiliente.
- **Résilience** : l'introduction d'un mécanisme de *dead-letter topic* pour les messages non traités, couplé à une bibliothèque de résilience telle que Resilience4j (disjoncteurs, tentatives), renforcerait la tolérance aux pannes.
- **Persistance vectorielle** : le remplacement de l'index en mémoire par une base vectorielle persistante, telle que l'extension pgvector de PostgreSQL, améliorerait la scalabilité de la base de connaissances.
- **Expérience conversationnelle** : la diffusion en flux (*streaming*) des réponses de l'agent offrirait une expérience utilisateur plus fluide, à l'image des interfaces conversationnelles modernes.
- **Notifications temps réel** : l'exploitation des événements Kafka pour alimenter des notifications en temps réel, via WebSocket, enrichirait l'interactivité de la plateforme.
- **Observabilité** : la mise en place d'une supervision centralisée (traces distribuées, métriques, journalisation agrégée) faciliterait l'exploitation et le diagnostic en production.

En définitive, SmartWealth AI constitue une base solide et extensible, qui démontre la faisabilité et la pertinence d'une plateforme de gestion de patrimoine à architecture micro-services augmentée par l'intelligence artificielle. Les perspectives esquissées ouvrent la voie à une solution toujours plus robuste, intelligente et proche des attentes des utilisateurs.

Code source : l'intégralité du code source du projet est disponible publiquement sur le dépôt GitHub suivant : https://github.com/moussafia/SmartWealth_AI

Webographie

- Documentation officielle Spring Boot — <https://spring.io/projects/spring-boot>
- Documentation officielle Spring Cloud — <https://spring.io/projects/spring-cloud>
- Documentation officielle Spring Cloud Gateway — <https://spring.io/projects/spring-cloud-gateway>
- Documentation officielle Apache Kafka — <https://kafka.apache.org/documentation/>
- Documentation officielle Keycloak — <https://www.keycloak.org/documentation>
- Documentation officielle LangChain4j — <https://docs.langchain4j.dev/>
- Documentation officielle Spring AI — <https://docs.spring.io/spring-ai/reference/>
- Documentation officielle Angular — <https://angular.dev/>
- Documentation officielle PostgreSQL — <https://www.postgresql.org/docs/>
- API CoinGecko — <https://www.coingecko.com/en/api>
- Spécification JSON Web Token (JWT) — <https://jwt.io/>
- Standard OAuth 2.0 — <https://oauth.net/2/>
- Dépôt GitHub du projet SmartWealth AI — https://github.com/moussafia/SmartWealth_AI